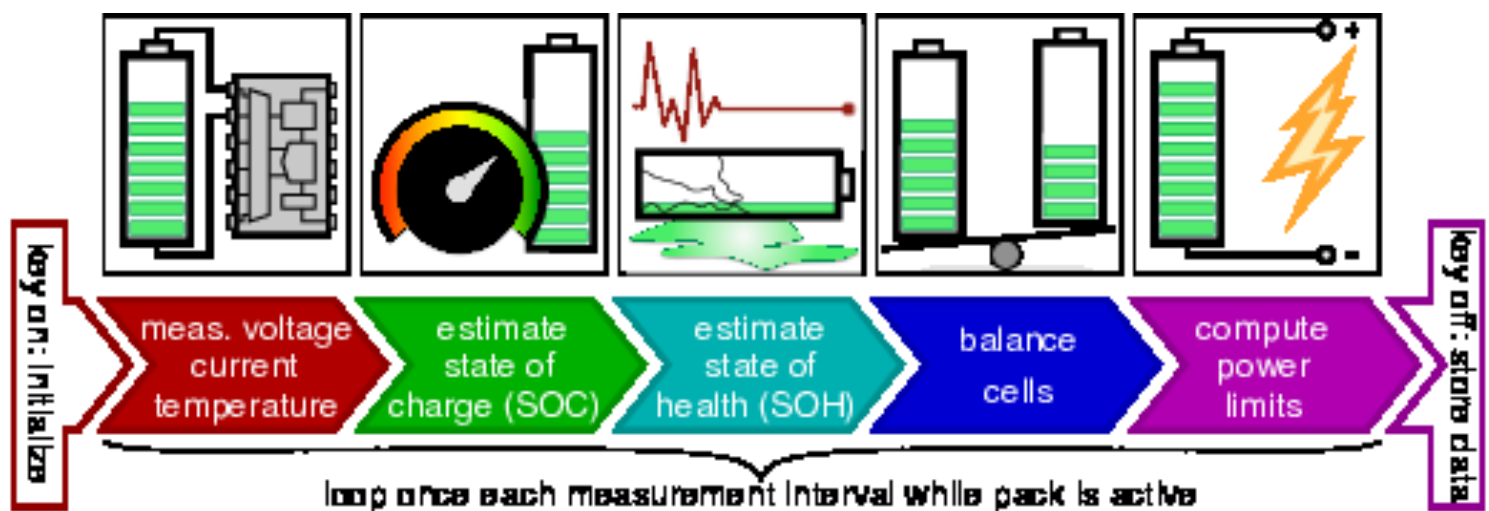


Battery Health Estimation

4.1: Introduction

- We know that the battery management system must estimate certain quantities indicative of the cells' states and parameters.
- We have now seen a number of ways to estimate battery cell state—the quickly changing quantities.
- Now, we turn our attention toward estimating battery cell parameters—the slowly changing, pseudo-static quantities.
- In particular, we are most interested in those quantities that reflect a change in the performance that the pack can deliver.
- These are indicators of battery pack “state-of-health” (SOH).



- Battery-pack health is most often summarized in terms of the present total capacity and present equivalent series resistance.

Total capacity

- As a battery cell ages, its total capacity decreases.
 - This is primarily due to unwanted side reactions and structural deterioration.
- This phenomena is often referred to as capacity fade.
- An up-to-date knowledge of total capacity is important because:
 - It is a major contributing factor to energy calculation;
 - It is a major contributing factor to SOC estimation if coulomb counting is used; else it is a minor contributing factor;
 - It does not have a significant role in power estimation.

Equivalent series resistance (ESR)

- As a battery cell ages, its equivalent series resistance increases.
 - This is also primarily due to unwanted side reactions and structural deterioration.
- An up-to-date knowledge of ESR is important because:
 - It is a major contributing factor to the power calculation;
 - It is a major contributing factor to SOC estimation for some voltage-based methods (*e.g.*, Tino); else, it is a minor factor;
 - It does not have a significant role in energy estimation.
- Since resistance and power are so tightly coupled, resistance rise in a cell is commonly referred to as power fade.

Other cell parameters

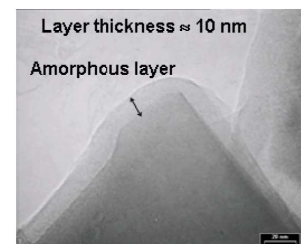
- The OCV relationship changes as the electrodes lose capacity.
 - For the cell chemistries in common use, it is unlikely that this change is large enough to be readily detectable.
- Other cell parameters almost certainly change as well; however, I'm not aware of BMS that make efforts to estimate them.
 - ESR and total capacity have the dominant impact;
 - Some KF-based methods in this chapter could be used to estimate the others, if the application demands it.
- In this chapter, we first look at some qualitative explanations for cell aging.
- Next, we explore a simple method to estimate ESR.
- We then look at KF-based methods to estimate up-to-date values for any desired set of cell parameters.
- We finally look at ways to estimate total capacity.

4.2: Lithium-ion aging: Negative electrode

- In the next sections, we'll seek to describe aging qualitatively.¹
- In the negative electrode, aging effects are seen at three scales:
 - At the surface of the electrode particles;
 - Within the electrode particles themselves;
 - Within the composite electrode structure (active materials; conductive additives; binder; current collector; porosity; etc).

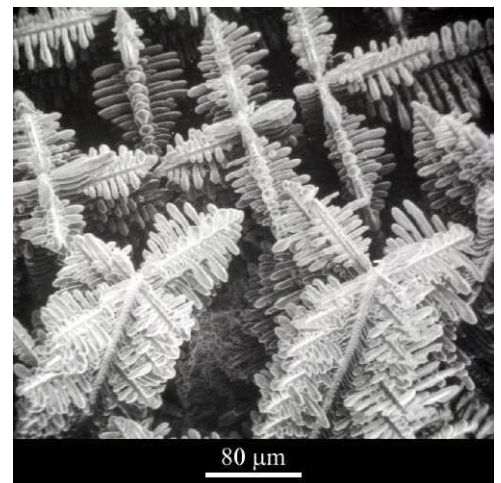
Negative electrode aging at surface of particles

- Here, we assume that a graphitic carbon is used as the negative electrode active material.
- Graphitic negative electrodes operate at voltages that are outside the electrochemical stability window of the electrolyte components.
- Reductive electrolyte decomposition takes place at the electrode/electrolyte interface when the electrode is in the charged state.
 - This process occurs mainly—but not exclusively—at the beginning of cycling, especially during the first “formation” cycle.
- The decomposition products form a “solid-electrolyte interphase” (SEI) surface film covering the electrode's surface.
- The SEI is a passivating layer, which slows/prevents further reaction between graphite and electrolyte.

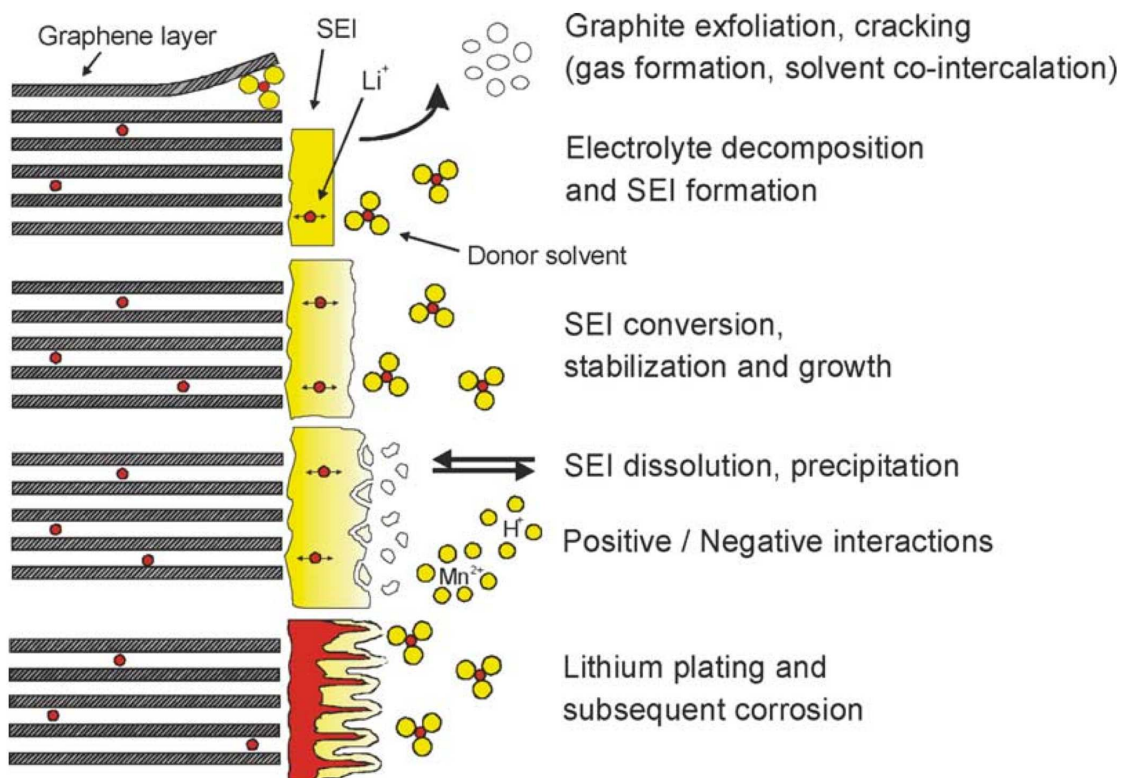


¹ Much of this is from: Vetter *et al.*, “Ageing mechanisms in lithium-ion batteries,” *Journal of Power Sources*, 147, 2005, 269–281.

- Lithium is consumed when SEI forms, lowering capacity of cell;
- SEI film is porous, allowing de/intercalation of lithium from/to graphite, but increases resistance of ion transfer.
- The composition of the SEI is unknown, and probably not uniform.
- It is suspected that numerous products form, then decompose, and form more stable products.
- High temperatures contribute to the breakdown of the SEI, which can lead to new SEI forming on exposed graphite.
- Pores in the SEI also allow some solvent to penetrate to the graphite surface, reacting and growing more SEI film.
- Trace water in electrolyte combines with ionized fluorine to form hydrofluoric acid (HF), which attacks the SEI—new SEI forms.
- Products of positive-electrode degradation also end up as part of SEI:
 - When not electrically conductive, cause increased resistance, and
 - Can plug pores, preventing lithium cycling, causing capacity fade.
- At low temperatures, diffusion in the particles is slower:
 - If charging is forced, local overpotential can attain level that causes lithium plating on particle surface;
 - Capacity irreversibly lost; dendrites can form and grow, eventually leading to internal short circuit.

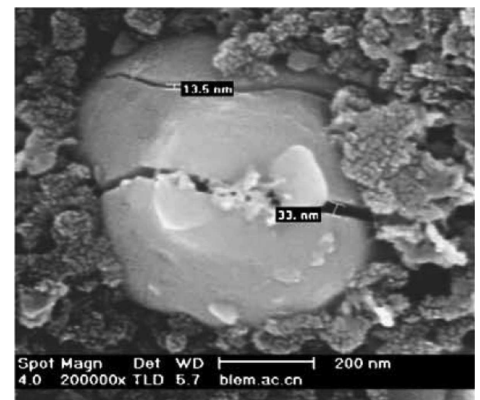


- Summary of the surface effects (from Vetter *et al.*):



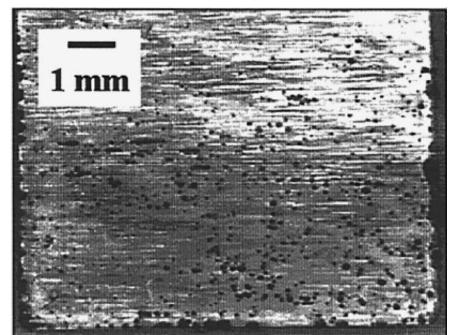
Negative electrode aging in bulk

- Dis/charging cell leads to small (anisotropic) volume changes in particles (usually less than 10 %), causing stress.
 - Can lead to cracking of particles, more SEI formation on exposed graphite;
 - Cracking of SEI itself, and more SEI formation on exposed graphite.
- Graphite exfoliation (layers flaking off) due to solvent intercalation with lithium is considered to have a bigger impact.
- Also, gasses released by solvent reaction with graphite inside the particles can accelerate cracking.



Negative electrode aging in composite electrode

- Stresses and strains within electrodes can cause mechanical and electronic contact loss:
 - Between graphite particles;
 - Between current collector and particles;
 - Between binder and particles;
 - Between binder and current collector.
- Results in higher impedance and can result in capacity loss if particles become electrically disconnected from current collector.
- Porosity of electrode can be reduced by volume changes and growth of SEI, impeding movement of lithium ions in electrolyte, increasing resistance.
- At low voltages (near 1.5 V) copper current collector can corrode, releasing Cu^{2+} into electrolyte:
 - Reduced current-collector/particle contact, higher cell resistance;
 - Corrosion products that deposit on electrode particles have poor electronic conductivity, giving higher film resistance;
 - Can lead to inhomogeneous current and potential distributions across cell plate area, leading to accelerated aging in parts of the cell, and preference toward lithium plating.
 - Copper also makes a metallic annealing site that can accelerate lithium plating, dendrite growth, and hence short circuits.



Summary of aging at negative electrode

Principal aging mechanisms at negative electrode (from Vetter *et al.*). **Boldface = considered more serious.**

Cause	Effect	Leads to	Enhanced by
Electrolyte decomposition, builds SEI, continuous low-rate reaction	Loss of lithium, impedance rise	Capacity fade, Power fade	High temperatures, high SOC (low potential)
Solvent co-intercalation, gas evolution and subsequent cracking formation in particles	Loss of active material (graphite exfoliation), loss of lithium	Capacity fade	Overcharge
Decrease of accessible surface area due to continuous SEI growth	Impedance rise	Power fade	High temperatures, high SOC (low potential)
Changes in porosity due to volume changes, SEI formation and growth	Impedance rise, larger overpotentials	Power fade	High cycling rate, high SOC (low potential)
Contact loss of active material particles due to volume changes during cycling	Loss of active material	Capacity fade	High cycling rate, low SOC
Decomposition of binder	Loss of lithium, loss of mechanical stability	Capacity fade	High SOC (low potential), high temperatures
Current collector corrosion	Larger overpotentials, impedance rise, Inhomogeneous distribution of current and potential	Power fade, Enhances other aging mechanisms	Overdischarge, low SOC (high potential)
Metallic lithium plating and subsequent electrolyte decomposition by metallic lithium	Loss of lithium (loss of electrolyte)	Capacity fade (power fade)	Low temperature, high cycling rates, poor cell balance, geometric misfits

4.3: Lithium ion aging: Positive electrode

- As with the negative electrode, aging occurs in three locations:
 - At the positive-electrode particle surface;
 - Within the active materials themselves;
 - In the bulk positive electrode.

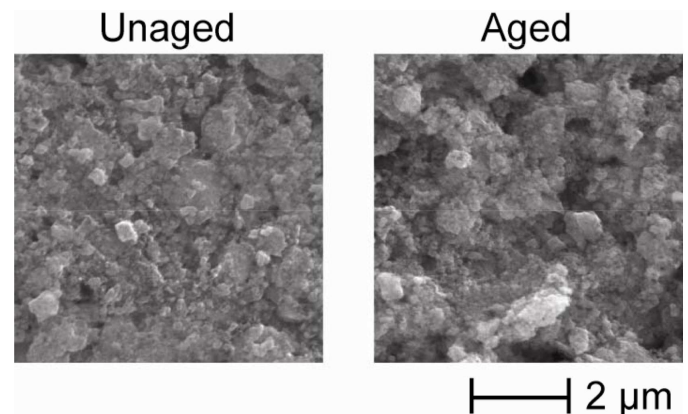
Positive electrode aging at surface of particles

- Electrolyte oxidation and LiPF_6 decomposition can form surface layer on positive electrode materials as well.
 - This is not as pronounced as for negative electrodes.
- A bigger factor is the dissolution of metals from the electrode into the electrolyte, and products formed from these metals which can re-precipitate on the surface as high-resistance film.
- Dissolution of Mn or Co into electrolyte results in capacity loss (fewer lithium storage sites), can poison negative electrode.
 - Mechanism depends on which oxide is used, but tends to happen predominantly at low/high states of charge, and can be greatly accelerated by high temperature.

Positive electrode aging in bulk

- Phase transitions (distortions in shape of crystal structure, without changing the structure itself) causes strains that can lead to cracking.
 - Transitions are caused by presence/absence of lithium in the storage sites, leading to different local molecular forces;

- Some phase transitions are normal and reversible;
 - Others lead to collapse (*e.g.*, some layered positive electrode structures on overcharge) and rapid capacity decrease.
- Can also lead to “structural disordering” when crystal structure of electrode breaks down (bonds are broken, and reform to different atoms, collapsing the tunnel-like structures that allow lithium movement, and lithium sites are lost (and, lithium can be trapped)).
 - Phase transitions near the surface can lead to permanent sub-surface layers forming that do not allow lithium to move as freely as in the unaltered crystal structure.
 - Some materials (*e.g.*, LFP) have been observed to have growing grain sizes as particles apparently sinter together.
 - This results in less surface area, and higher resistance.



Positive electrode aging in composite electrode

- The positive electrode also experiences degradation of the inactive components of the cell:
 - Binder decomposition;
 - Oxidation of conductive particles (*e.g.*, carbon black);
 - Corrosion of the current collector;
 - Loss of contact to conductive particles due to volume changes.

Summary of aging at positive electrode

Principal aging mechanisms at positive electrode (adapted from Vetter *et al.*).

Cause	Effect	Leads to	Enhanced by
Phase transitions	Cracking of active particles	Capacity fading	High rates, high/low SOC
Structural disordering	Lithium sites lost and lithium trapped	Capacity fading	High rates, high/low SOC
Metal dissolution and/or electrolyte decomposition	Migration of soluble species,	Capacity fading	High/low SOC, high temperature
	Reprecipitation of new phases,	Power fade	
	Surface layer formation	Power fade	
Electrolyte decomposition	Gas evolution		High temperature
Binder decomposition	Loss of contact	Power fade	
Oxidation of conductive agent	Loss of contact	Power fade	
Corrosion of current collector	Loss of contact	Power fade	High SOC

4.4: Sensitivity of voltage to ESR and total capacity

Sensitivity to ESR

- Estimating ESR turns out to be relatively simple because it is highly “observable” from voltage measurements.

- Consider $v_k = \text{OCV}(z_k) + M h_k - \sum_i R_i i_{R_i,k} - i_k R_0$.

- Define the sensitivity of the voltage measurement to a change in resistance as

$$S_{v_k}^{R_0} = \frac{R_0}{v_k} \frac{dv_k}{dR_0} = \frac{-R_0}{v_k} i_k.$$

- Since i_k can be very large, the absolute sensitivity is high.
- One approach to estimating R_0 is to compare voltages at two adjacent time samples

$$\begin{aligned} v_k &= \text{OCV}(z_k) + M h_k - \sum_i R_i i_{R_i,k} - i_k R_0 \\ v_{k-1} &= \text{OCV}(z_{k-1}) + M h_{k-1} - \sum_i R_i i_{R_i,k-1} - i_{k-1} R_0 \end{aligned}$$

$$v_k - v_{k-1} \approx R_0 (i_{k-1} - i_k),$$

where we observe that SOC, v_{C_i} and h_k change relatively slowly compared to how quickly i_k changes.

- So, we can estimate

$$\widehat{R}_{0,k} = \frac{v_k - v_{k-1}}{i_{k-1} - i_k}.$$

ISSUE: Can compute $\widehat{R}_{0,k}$ only when $\Delta i_k \neq 0$. So, we skip updates when $|\Delta i_k|$ is small (avoids amplification of noise, as well).

- Because of the inaccuracy of the ESC model (imperfect fidelity with respect to the true cell behavior) and inaccuracy introduced via specific approximations, $\widehat{R}_{0,k}$ is quite noisy.

- Might consider using total least squares approach (see later re. total capacity estimation) but can also simply filter. For example,

$$\widehat{R}_{0,k}^{\text{filt}} = \alpha \widehat{R}_{0,k-1}^{\text{filt}} + (1 - \alpha) \widehat{R}_{0,k},$$

where $0 \ll \alpha < 1$.

- Tends to work quite well.

ISSUE: ESR is SOC dependent.

- SOC dependence could be handled by adapting resistance vectors rather than scalars.

ISSUE: ESR is temperature dependent.

- Temperature dependence can be well modeled as

$$R_0 = R_{0,\text{ref}} \exp \left(E_{R_0,\text{ref}} \left(\frac{1}{T_{\text{ref}}} - \frac{1}{T} \right) \right),$$

but if pack dwells near one temperature for an extended period, results at other temperatures may become biased.

- Can also be handled by using adaptive matrix of resistances vs. SOC and temperature.

Sensitivity to total capacity

- Estimating total capacity turns out to be quite difficult.
- Consider the sensitivity of the voltage measurement to capacity:

$$S_{v_k}^Q = \frac{Q}{v_k} \frac{dv_k}{dQ} = \frac{Q}{v_k} \frac{d}{dQ} \left(\text{OCV}(z_k) + M h_k - \sum_i R_i i_{R_i,k} - i_k R_0 \right).$$

- Consider the first term:

$$\frac{d\text{OCV}(z_k)}{dQ} = \frac{\partial \text{OCV}(z_k)}{\partial z_k} \frac{dz_k}{dQ}.$$

- For most cells, the slope of the OCV curve is very shallow, so $\partial \text{OCV}(z_k) / \partial z_k$ is very small. Further,

$$\begin{aligned} \frac{dz_k}{dQ} &= \frac{dz_{k-1}}{dQ} - \eta_{k-1} i_{k-1} \Delta t \frac{d(1/Q)}{dQ} \\ &= \frac{dz_{k-1}}{dQ} + \frac{\eta_{k-1} i_{k-1} \Delta t}{Q^2}. \end{aligned}$$

- The first term can be calculated recursively.
 - It grows when i_k is in the same direction for a considerable amount of time and shrinks when i_k changes direction.
 - For random i_k (e.g., HEV) it is around the same order of magnitude as the second term.
- The second term has a Δt factor in it, which is often on the order of $1/3600$, which is quite small.
- So, sensitivity of voltage to capacity through the OCV term is small.
- Similarly, the sensitivity of the voltage to capacity through the hysteresis term is small (it is zero through the other terms).
- As a consequence, individual voltage measurements have very little information regarding capacity.
- Must somehow combine many voltage measurements.
- Simple ideas, like used to estimate $\widehat{R}_0$ will not work well.
- So, we explore two basic approaches:
 - First, look at KF-based approaches, which can work well;
 - Next, look at total-least-squares approaches, which are optimal.

4.5: A Kalman filter framework for estimating parameters

- We know that Kalman filters may be used to estimate the state of a dynamic system given known parameters and noisy measurements.
- We may also use (nonlinear) Kalman filters to estimate parameters given a known state and noisy measurements.
- In this section of notes we first consider how to estimate the parameters of a system if its state is known.
- Next, we consider how to estimate both the state and parameters of the system simultaneously using two different approaches.

A generic approach to parameter estimation

- We denote the true parameters of a particular model by θ .
- We will use Kalman-filtering techniques to estimate the parameters much like we have estimated the state. Therefore, we require a model of the dynamics of the parameters.
- By assumption, parameters change very slowly, so we model them as constant with some small perturbation:

$$\theta_k = \theta_{k-1} + r_{k-1}.$$

- The small white noise input r_k is fictitious, but models the slow drift in the parameters of the system plus the infidelity of the model structure.
- The output equation required for Kalman-filter system identification must be a measurable function of the system parameters. We use

$$d_k = h(x_k, u_k, \theta, e_k),$$

where $h(\cdot)$ is the output equation of the system model being identified, and e_k models the sensor noise and modeling error.

- Note that d_k is usually the same measurement as y_k , but we maintain a distinction here in case separate outputs are used.
 - Then, $\mathbb{D}_k = \{d_0, d_1, \dots, d_k\}$.
 - Also, note that e_k and v_k often play the same role, but are also considered distinct here.
- We also slightly revise the mathematical model of system dynamics

$$x_k = f(x_{k-1}, u_{k-1}, \theta, w_{k-1})$$

$$y_k = h(x_k, u_k, \theta, v_k),$$

to explicitly include the parameters θ in the model.

- Non-time-varying numeric values required by the model may be embedded within $f(\cdot)$ and $h(\cdot)$, and are not included in θ .

SPKF for parameter estimation

- Parameter estimation with SPKF is relatively straightforward, so we discuss it before we discuss EKF.
- We first define an augmented random vector θ^a that combines the randomness of the parameters and sensor noise. This augmented vector is used in the estimation process as described below.
- As always, we proceed by deriving the six essential steps of sequential probabilistic inference.

SPKF step 1a: Parameter prediction time update.

- The parameter prediction step is approximated as

$$\hat{\theta}_k^{a,-} = \mathbb{E}[\theta_{k-1}^a + r_{k-1} \mid \mathbb{D}_{k-1}] = \hat{\theta}_{k-1}^{a,+}.$$

- This makes sense, since the parameters are assumed constant.

SPKF step 1b: Error covariance time update.

- The covariance prediction step is accomplished by first computing $\tilde{\theta}_k^-$.

$$\begin{aligned}\tilde{\theta}_k^{a,-} &= \theta_k^a - \hat{\theta}_k^{a,-} = \theta_{k-1}^a + r_k - \hat{\theta}_{k-1}^{a,+} \\ &= \tilde{\theta}_{k-1}^{a,+} + r_k.\end{aligned}$$

- We then directly compute the desired covariance

$$\begin{aligned}\Sigma_{\tilde{\theta},k}^{a,-} &= \mathbb{E}[\tilde{\theta}_k^{a,-}(\tilde{\theta}_k^{a,-})^T] = \mathbb{E}[(\tilde{\theta}_{k-1}^{a,+} + r_k)(\tilde{\theta}_{k-1}^{a,+} + r_k)^T] \\ &= \Sigma_{\tilde{\theta},k-1}^{a,+} + \Sigma_{\tilde{r}}.\end{aligned}$$

- The time-updated covariance has additional uncertainty due to the fictitious noise “driving” the parameter values.

SPKF step 1c: Predict system output d_k .

- To predict the system output, we require a set of sigma points describing the output.
- This in turn requires a set of $p + 1$ sigma points describing $\theta_k^{a,-}$, which we will denote as $\mathcal{W}_k^{a,-}$.

$$\mathcal{W}_k^{a,-} = \left\{ \hat{\theta}_k^{a,-}, \hat{\theta}_k^{a,-} + \gamma \sqrt{\Sigma_{\tilde{\theta},k}^{a,-}}, \hat{\theta}_k^{a,-} - \gamma \sqrt{\Sigma_{\tilde{\theta},k}^{a,-}} \right\}.$$

- From the augmented sigma points, the $p + 1$ vectors comprising the parameters portion $\mathcal{W}_k^{\theta,-}$ and the $p + 1$ vectors comprising the modeling error portion $\mathcal{W}_k^{e,-}$ are extracted.
- The output equation is evaluated using all pairs of $\mathcal{W}_{k,i}^{\theta,-}$ and $\mathcal{W}_{k,i}^{e,-}$ (where the subscript i denotes that the i th vector is being extracted from the original set), yielding the sigma points $\mathcal{D}_{k,i}$ for time step k .

- That is, $\mathcal{D}_{k,i} = h(x_k, u_k, \mathcal{W}_{k,i}^{\theta,-}, \mathcal{W}_{k,i}^{e,-})$.
- Finally, the output prediction is computed as

$$\begin{aligned}\hat{d}_k^- &= \mathbb{E}[h(x_k, u_k, \theta, e_k) \mid \mathbb{D}_{k-1}] \\ &\approx \sum_{i=0}^p \alpha_i^{(m)} h(x_k, u_k, \mathcal{W}_{k,i}^{\theta,-}, \mathcal{W}_{k,i}^{e,-}) = \sum_{i=0}^p \alpha_i^{(m)} \mathcal{D}_{k,i}.\end{aligned}$$

SPKF step 2a: Estimator gain matrix L_k^θ .

- To compute the estimator gain matrix, we must first compute the required covariance matrices.

$$\begin{aligned}\Sigma_{\tilde{d},k} &= \sum_{i=0}^p \alpha_i^{(c)} (\mathcal{D}_{k,i} - \hat{d}_k) (\mathcal{D}_{k,i} - \hat{d}_k)^T \\ \Sigma_{\tilde{\theta}\tilde{d},k}^- &= \sum_{i=0}^p \alpha_i^{(c)} (\mathcal{W}_{k,i}^{\theta,-} - \hat{\theta}_k^{a,-}) (\mathcal{D}_{k,i} - \hat{d}_k)^T.\end{aligned}$$

- Then, we simply compute $L_k^\theta = \Sigma_{\tilde{\theta}\tilde{d},k}^- \Sigma_{\tilde{d},k}^{-1}$.

SPKF step 2b: Parameter estimate measurement update.

- The fifth step is to compute the *a posteriori* parameter estimate by updating the *a priori* prediction using the estimator gain and the output prediction error $d_k - \hat{d}_k$

$$\hat{\theta}_k^{a,+} = \hat{\theta}_k^{a,-} + L_k^\theta (d_k - \hat{d}_k).$$

SPKF step 2c: Error covariance measurement update.

- The final step is calculated directly from the optimal formulation:

$$\Sigma_{\tilde{\theta},k}^{a,+} = \Sigma_{\tilde{\theta},k}^{a,-} - L_k^\theta \Sigma_{\tilde{d},k} (L_k^\theta)^T.$$

4.6: EKF for parameter estimation

- Here, we show how to use EKF for parameter estimation.

EKF step 1a: Parameter prediction time update.

- Due to the linearity of the parameter dynamics equation, we have $\hat{\theta}_k^- = \hat{\theta}_{k-1}^+$ (same as for SPKF).

EKF step 1b: Error covariance time update.

- Again, due to the linearity of the parameter dynamics equation, we have $\Sigma_{\tilde{\theta},k}^- = \Sigma_{\tilde{\theta},k-1}^+ + \Sigma_{\tilde{r},k-1}$ (same as for SPKF).

EKF step 1c: Output prediction.

- The system output is predicted to be

$$\begin{aligned}\hat{d}_k &= \mathbb{E}[h(x_k, u_k, \theta, e_k) \mid \mathbb{D}_{k-1}] \\ &\approx h(x_k, u_k, \hat{\theta}_k^-, \bar{e}_k).\end{aligned}$$

- That is, it is assumed that propagating $\hat{\theta}_k^-$ and the mean estimation error is the best approximation to predicting the output.

EKF step 2a: Estimator gain matrix.

- The output prediction error may then be approximated

$$\begin{aligned}\tilde{d}_k &= d_k - \hat{d}_k \\ &= h(x_k, u_k, \theta, e_k) - h(x_k, u_k, \hat{\theta}_k^-, \bar{e}_k)\end{aligned}$$

using again a Taylor-series expansion on the first term.

$$d_k \approx h(x_k, u_k, \hat{\theta}_k^-, \bar{e}_k)$$

$$\begin{aligned}
& + \underbrace{\left. \frac{dh(x_k, u_k, \theta, e_k)}{d\theta} \right|_{\theta=\hat{\theta}_k^-}}_{\text{Defined as } \hat{C}_k^\theta} (\theta - \hat{\theta}_k^-) \\
& + \underbrace{\left. \frac{dh(x_k, u_k, \theta, e_k)}{de_k} \right|_{e_k=\bar{e}_k}}_{\text{Defined as } \hat{D}_k^\theta} (e_k - \bar{e}_k).
\end{aligned}$$

- From this, we can compute such necessary quantities as

$$\begin{aligned}
\Sigma_{\tilde{d},k} & \approx \hat{C}_k^\theta \Sigma_{\tilde{\theta},k}^- (\hat{C}_k^\theta)^T + \hat{D}_k^\theta \Sigma_{\tilde{e}} (\hat{D}_k^\theta)^T, \\
\Sigma_{\tilde{\theta},k}^- & \approx \mathbb{E}[(\tilde{\theta}_k^-)(\hat{C}_k^\theta \tilde{\theta}_k^- + \hat{D}_k^\theta \tilde{e}_k)^T] \\
& = \Sigma_{\tilde{\theta},k}^- (\hat{C}_k^\theta)^T.
\end{aligned}$$

- These terms may be combined to get the Kalman gain

$$L_k^\theta = \Sigma_{\tilde{\theta},k}^- (\hat{C}_k^\theta)^T [\hat{C}_k^\theta \Sigma_{\tilde{\theta},k}^- (\hat{C}_k^\theta)^T + \hat{D}_k^\theta \Sigma_{\tilde{e}} (\hat{D}_k^\theta)^T]^{-1}.$$

- Note, by the chain rule of total differentials,

$$\begin{aligned}
dh(x_k, u_k, \theta, e_k) & = \frac{\partial h(x_k, u_k, \theta, e_k)}{\partial x_k} dx_k + \frac{\partial h(x_k, u_k, \theta, e_k)}{\partial u_k} du_k + \\
& \quad \frac{\partial h(x_k, u_k, \theta, e_k)}{\partial \theta} d\theta + \frac{\partial h(x_k, u_k, \theta, e_k)}{\partial e_k} de_k
\end{aligned}$$

$$\begin{aligned}
\frac{dh(x_k, u_k, \theta, e_k)}{d\theta} & = \frac{\partial h(x_k, u_k, \theta, e_k)}{\partial x_k} \frac{dx_k}{d\theta} + \frac{\partial h(x_k, u_k, \theta, e_k)}{\partial u_k} \underbrace{\frac{du_k}{d\theta}}_0 + \\
& \quad \frac{\partial h(x_k, u_k, \theta, e_k)}{\partial \theta} \frac{d\theta}{d\theta} + \frac{\partial h(x_k, u_k, \theta, e_k)}{\partial e_k} \underbrace{\frac{de_k}{d\theta}}_0 \\
& = \frac{\partial h(x_k, u_k, \theta, e_k)}{\partial \theta} + \frac{\partial h(x_k, u_k, \theta, e_k)}{\partial x_k} \frac{dx_k}{d\theta}.
\end{aligned}$$

- But,

$$\frac{dx_k}{d\theta} = \frac{\partial f(x_{k-1}, u_{k-1}, \theta, w_{k-1})}{\partial \theta} + \frac{\partial f(x_{k-1}, u_{k-1}, \theta, w_{k-1})}{\partial x_{k-1}} \frac{dx_{k-1}}{d\theta}.$$

- The derivative calculations are recursive in nature, and evolve over time as the state evolves.
- The term $dx_0/d\theta$ is initialized to zero unless side information gives a better estimate of its value.
- To calculate $\hat{C}_k^\theta$ for any specific model structure, we require methods to calculate all of the above partial derivatives for that model.

EKF step 2b: Parameter estimate measurement update.

- The fifth step is to compute the *a posteriori* parameter estimate by updating the *a priori* prediction using the estimator gain and the output prediction error $d_k - \hat{d}_k$

$$\hat{\theta}_k^+ = \hat{\theta}_k^- + L_k^\theta (d_k - \hat{d}_k).$$

EKF step 2c: Error covariance measurement update.

- Finally, the updated covariance is computed as

$$\Sigma_{\tilde{\theta},k}^+ = \Sigma_{\tilde{\theta},k}^- - L_k^\theta \Sigma_{\tilde{d},k} (L_k^\theta)^T.$$

- EKF for parameter estimation is summarized in a later table.

Notes:

- We initialize the parameter estimate with our best information re. the parameter value: $\hat{\theta}_0^+ = \mathbb{E}[\theta_0]$.
- We initialize the parameter estimation error covariance matrix:

$$\Sigma_{\tilde{\theta},0}^+ = \mathbb{E}[(\theta - \hat{\theta}_0^+)(\theta - \hat{\theta}_0^+)^T].$$

- We also initialize $dx_0/d\theta = 0$ unless side information is available.

4.7: Simultaneous state and parameter estimation

- We have now seen how to use Kalman filters to perform state estimation and parameter estimation independently.
- How about both at the same time?
- There are two approaches to doing so: Joint estimation and dual estimation. These are discussed in the next sections.

Generic joint estimation

- In joint estimation, the state vector and parameter vector are combined, and a Kalman filter simultaneously estimates the values of this augmented state vector.
- It has the disadvantages of large matrix operations due to the high dimensionality of the resulting augmented model and potentially poor numeric conditioning due to the vastly different time scales of the states (including parameters) in the augmented state vector.
- However, it is quite straightforward to implement. We first combine the state and parameter vectors to form augmented dynamics

$$\begin{bmatrix} x_k \\ \theta_k \end{bmatrix} = \begin{bmatrix} f(x_{k-1}, u_{k-1}, \theta_{k-1}, w_{k-1}) \\ \theta_{k-1} + r_{k-1} \end{bmatrix}$$

$$y_k = h(x_k, u_k, \theta_k, v_k).$$

- To simplify notation, let $\mathbb{X}_k$ be the augmented state, $\mathbb{W}_k$ be the augmented noise, and $\mathcal{F}$ be the augmented state equations:

$$\mathbb{X}_k = \mathcal{F}(\mathbb{X}_{k-1}, u_{k-1}, \mathbb{W}_{k-1})$$

$$y_k = h(\mathbb{X}_k, u_k, v_k).$$

- With the augmented model of the system state dynamics and parameter dynamics defined, we apply a nonlinear KF method.

Generic dual estimation

- In dual estimation, separate Kalman filters are used for state estimation and parameter estimation.
- The computational complexity is smaller and the matrix operations may be numerically better conditioned.
- However, by decoupling state from parameters, any cross-correlations between changes are lost, leading to potentially poorer accuracy.
- The mathematical model of state dynamics again explicitly includes the parameters as the vector θ_k :

$$x_k = f(x_{k-1}, u_{k-1}, w_{k-1}, \theta_{k-1})$$

$$y_k = h(x_k, u_k, v_k, \theta_{k-1}).$$

- Non-time-varying numeric values required by the model may be embedded within $f(\cdot)$ and $h(\cdot)$, and are not included in θ_k .
- We also slightly revise the mathematical model of parameter dynamics to include the effect of the state equation explicitly.

$$\theta_k = \theta_{k-1} + r_{k-1}$$

$$d_k = h\left(f(x_{k-1}, u_{k-1}, \bar{w}_{k-1}, \theta_{k-1}), u_k, e_k, \theta_{k-1}\right).$$

- The dual filters can be viewed by drawing a block diagram. (The interactions will be made clearer later)

- We see that the process essentially comprises two Kalman filters running in parallel—one adapting the state and one adapting parameters—with some information exchange between the filters.

Joint state and parameter estimation via EKF

- Applying EKF to the joint estimation problem is straightforward. But, don't forget the recursive calculation of $d\mathcal{F}/d\mathbb{X}$.

Dual state and parameter estimation via EKF

- Two EKFs are implemented, with their signals mixed.
- Again, we need to be careful when computing $\hat{C}_k^\theta$, which requires a total-differential expansion to be correct

$$\hat{C}_k^\theta = \left. \frac{dg(\hat{x}_k^-, u_k, \theta)}{d\theta} \right|_{\theta=\hat{\theta}_k^-}$$

$$\frac{dg(\hat{x}_k^-, u_k, \theta)}{d\theta} = \frac{\partial g(\hat{x}_k^-, u_k, \theta)}{\partial \theta} + \frac{\partial g(\hat{x}_k^-, u_k, \theta)}{\partial \hat{x}_k^-} \frac{d\hat{x}_k^-}{d\theta}$$

$$\frac{d\hat{x}_k^-}{d\theta} = \frac{\partial f(\hat{x}_{k-1}^+, u_{k-1}, \theta)}{\partial \theta} + \frac{\partial f(\hat{x}_{k-1}^+, u_{k-1}, \theta)}{\partial \hat{x}_{k-1}^+} \frac{d\hat{x}_{k-1}^+}{d\theta}$$

$$\frac{d\hat{x}_{k-1}^+}{d\theta} = \frac{d\hat{x}_{k-1}^-}{d\theta} - L_{k-1}^x \frac{dg(\hat{x}_{k-1}^-, u_{k-1}, \theta)}{d\theta},$$

- This assumes that L_{k-1}^x is not a function of θ . (It is—weakly—but it's not worth the extra computation to consider it so.)
- The 3 total derivatives are computed recursively, initialized to zero.

Joint state and parameter estimation via SPKF

- Uses a standard SPKF where state vector is augmented with params.

Dual state and parameter estimation via SPKF

- This, just like dual estimation using EKF, uses two filters. Both employ the SPKF algorithm and intermix signals.

4.8: Robustness and speed

Ensuring correct convergence

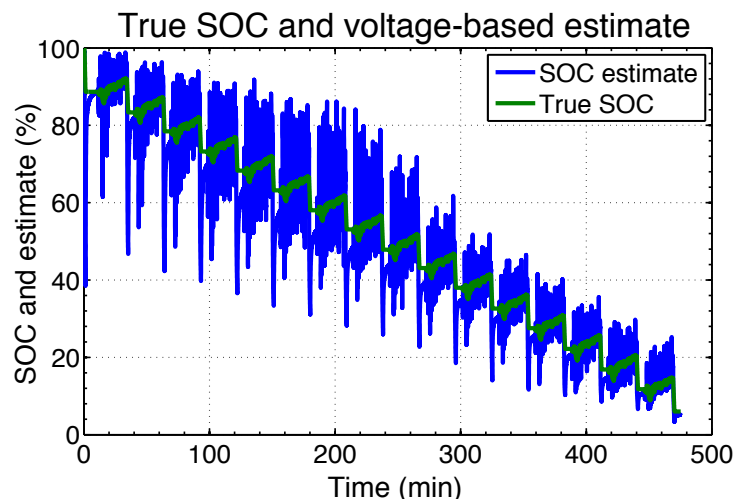
- Dual and joint filtering adapt $\hat{x}$ and $\hat{\theta}$ so that the model input–output relationship matches system’s input–output data closely.
- There is no built-in guarantee that the state of the model converges to anything with physical meaning.
- Usually, when employing a Kalman filter, we are concerned that the state converge to a very specific meaning.
- Special steps must be taken to ensure that this occurs.
- First, a very crude cell model may be used, combined with the dual/joint EKF/SPKF to ensure convergence of the SOC state.
- Specifically, the cell terminal voltage

$$v_k \approx \text{OCV}(z_k) - R_0 i_k$$

$$\text{OCV}(z_k) \approx v_k + R_0 i_k$$

$$\hat{z}_k = \text{OCV}^{-1}(v_k + R_0 i_k).$$

- By measuring the cell voltage under load, v_k , the cell current i_k , and having knowledge of R_0 , and knowing the inverse OCV function for the cell chemistry, one can compute a noisy estimate of SOC, $\hat{z}_k$.



- The cell model being used in the KF has its output equation augmented with SOC. For example,

$$g(x_k, u_k, \theta) = \begin{bmatrix} \text{OCV}(z_k) + M h_k - \sum_i R_i i_{R_i,k} - R_0 i_k \\ z_k \end{bmatrix}.$$

- The dual/joint xKF is run using this modified model, with the “measured” information used in the measurement update being

$$y_k = \begin{bmatrix} v_k \\ \hat{z}_k \end{bmatrix}.$$

- While the “noise” of $\hat{z}_k$ (short-term bias due to hysteresis effects and polarization filter voltages being ignored) prohibit it from being used as the primary estimator of SOC, its expected long-term behavior in a dynamic environment is accurate, and maintains the accuracy of the SOC state in the dual/joint xKF.

Methods for estimating SOH without a full dual EKF/SPKF

- The full dual/joint EKF/SPKF method is computationally expensive.
- If precise values for the full set of cell-model parameters are not necessary, then other methods might be used.
- Here, we present methods to determine cell capacity and resistance using simpler KF-based methods.

Estimating resistance using a simple xKF

- To estimate cell resistance using KF, we formulate a simple model

$$R_{0,k+1} = R_{0,k} + r_k$$

$$v_k = \text{OCV}(z_k) - R_{0,k}i_k + e_k,$$

where $R_{0,k}$ is the cell resistance and is modeled as a constant value with a fictitious noise process r_k allowing adaptation.

- v_k is a crude estimate of the cell's voltage, i_k is the cell current, and e_k models estimation error.
- If an estimate of z_k is available from an external source, we simply apply KF to this model to estimate cell resistance.
- The above model may be extended to handle different values of resistance on dis/charge, or at different SOC's, or at different temperatures, for example.

Estimating capacity using a simple xKF

- To estimate cell capacity using KF, we formulate a simple cell model

$$Q_{k+1} = Q_k + r_k$$

$$d_k = z_k - z_{k-1} + \eta_{k-1}i_{k-1}\Delta t / Q_{k-1} + e_k.$$

- The second equation is a reformulation of the SOC state equation such that the expected value of d_k is equal to zero by construction.
- Again, a KF is constructed using the model defined by these two equations to produce a capacity estimate.
- As the KF runs, the computation for d_k in the second equation is compared to the known value (zero, by construction), and the difference is used to update the capacity estimate.
- Note that good estimates of the present and previous states-of-charge are required, possibly from a KF estimating SOC.

4.9: The problem with least-squares capacity estimates

- We now return to the issue of estimating cell total capacity.
- Recall that the sensitivity of the cell voltage to cell capacity is very low, so noise tends to bias results.
- The KF-based methods are able to minimize the impact of noise on the estimates, but even these are biased by noise, as we will see.
- Consider the SOC equation in continuous time:

$$z[k_2] = z[k_1] - \frac{1}{Q} \sum_{k=k_1}^{k_2-1} \eta[k] i[k].$$

- We can rearrange its terms to get:

$$\underbrace{- \sum_{k=k_1}^{k_2-1} \eta[k] i[k]}_y = Q \underbrace{(z[k_2] - z[k_1])}_x,$$

where the obvious linear structure of $y = Qx$ becomes apparent.

- Using a regression technique, for example, one may compute estimates of Q . One needs only to find values for “ x ” and “ y ”.
- The problem with using standard (least squares) linear regression techniques is that both the summed current value y and the difference between state-of-charge values x have sensor noise or estimation noise associated with them.
- The least squares linear regression problem is a solution to the equation $(y - \Delta y) = Qx$; that is, there is noise assumed on the measurements y , but not on the independent variable x .

- However, the total-capacity-estimation problem is implicitly of the form $(y - \Delta y) = Q(x - \Delta x)$ since both the integrated current and SOC estimates have noise.
- That is, because estimates of SOC are generally imperfect, there will be noise on the x variable, and using standard least squares linear regression results in an inaccurate and biased estimate of battery cell total capacity.
 - Note that KF-based methods are (recursive) least squares: they will tend to be biased by noise.
- The usual approach to counteract this problem is to try to ensure that the SOC estimates are as accurate as possible and then use standard least-squares estimation anyway.
- For example, we might put constraints on how the capacity is estimated.
 - We could force the cell current to be zero before the test begins and after the test ends (so that the cell is in an equilibrium state and the SOC estimates are as accurate as possible).
 - This procedure eliminates to a large extent (but not completely) the error in the x variable, and makes the regression reasonably accurate.
- This method still does not correctly handle the residual noise in x : while it minimizes the noise, it never totally eliminates it.
- The solution is to use “total least squares” instead of “(ordinary) least squares” estimation.

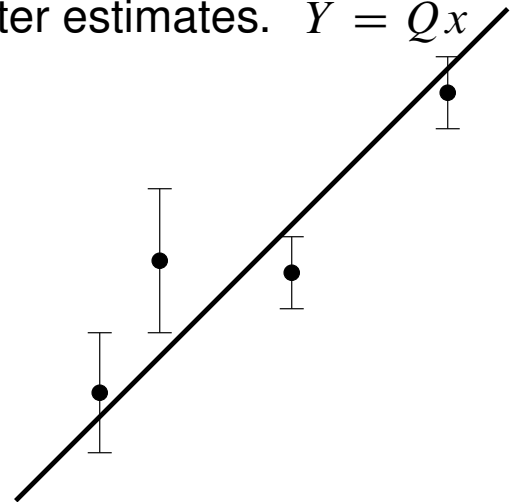
4.10: Derivation of weighted ordinary least squares

- Both ordinary least squares (OLS) and total least squares (TLS), as applied to battery cell total capacity estimation, seek to find a constant $\widehat{Q}$ such that $y \approx \widehat{Q}x$ using N -vectors of measured data $\mathbf{x}$ and $\mathbf{y}$.
- The i th element x_i in $\mathbf{x}$ and y_i in $\mathbf{y}$ correspond to data collected from a cell over an interval of time, where x_i is the estimated change in state-of-charge over that interval, and y_i is the accumulated ampere hours passing through the cell during that period.
- Specifically,

$$x_i = z[k_2] - z[k_1] \quad \text{for time interval } i$$

$$y_i = - \sum_{k=k_1}^{k_2-1} \eta[k]i[k].$$

- The vectors $\mathbf{x}$ and $\mathbf{y}$ must be at least one sample long ($N \geq 1$), but multiple samples may be used to obtain better estimates. $Y = \widehat{Q}x$
- The OLS approach assumes that there is no error on the x_i , and models the data as $\mathbf{y} = Q\mathbf{x} + \Delta\mathbf{y}$, where $\Delta\mathbf{y}$ is a vector of measurement errors.
- The error bars on the data point are meant to illustrate the uncertainties, which are proportional to σ_{y_i} .
- We assume that $\Delta\mathbf{y}$ comprises zero-mean Gaussian random variables, with known variances $\sigma_{y_i}^2$ (which are not necessarily equal to each other).



- OLS attempts to find an estimate $\widehat{Q}$ of the true cell total capacity Q that minimizes the sum of squared errors Δy_i .
- We generalize that approach here slightly to allow for finding a $\widehat{Q}$ that minimizes the sum of *weighted* squared errors, where the weighting takes into account the uncertainty of the measurement.
- That is, we desire to find a $\widehat{Q}$ that minimizes the weighted least squares (WLS) merit function

$$\chi_{\text{WLS}}^2 = \sum_{i=1}^N \frac{(y_i - Y_i)^2}{\sigma_{y_i}^2} = \sum_{i=1}^N \frac{(y_i - \widehat{Q}x_i)^2}{\sigma_{y_i}^2}.$$

- In this equation, Y_i is a point on the line $Y_i = \widehat{Q}x_i$ corresponding to the measured data pair (x_i, y_i) , where y_i is assumed to have noise but x_i has no noise.
- There are a number of approaches that may be taken to solve this problem, but one that will serve our purposes well is to differentiate the merit function with respect to $\widehat{Q}$ and solve for $\widehat{Q}$ by setting the partial derivative to zero.

$$\begin{aligned} \frac{\partial \chi_{\text{WLS}}^2}{\partial \widehat{Q}} &= -2 \sum_{i=1}^N \frac{x_i (y_i - \widehat{Q}x_i)}{\sigma_{y_i}^2} = 0 \\ \widehat{Q} \sum_{i=1}^N \frac{x_i^2}{\sigma_{y_i}^2} &= \sum_{i=1}^N \frac{x_i y_i}{\sigma_{y_i}^2} \\ \widehat{Q} &= \sum_{i=1}^N \frac{x_i y_i}{\sigma_{y_i}^2} \bigg/ \sum_{i=1}^N \frac{x_i^2}{\sigma_{y_i}^2}. \end{aligned}$$

- If we define

$$c_{1,n} = \sum_{i=1}^n \frac{x_i^2}{\sigma_{y_i}^2}, \quad \text{and} \quad c_{2,n} = \sum_{i=1}^n \frac{x_i y_i}{\sigma_{y_i}^2},$$

then we can write $\widehat{Q}_n = c_{2,n}/c_{1,n}$.

- The two quantities $c_{1,n}$ and $c_{2,n}$ may be computed recursively to minimize storage requirements and to even out computational requirements when updating $\widehat{Q}_n$ when n gets large

$$c_{1,n} = c_{1,n-1} + x_n^2/\sigma_{y_n}^2$$

$$c_{2,n} = c_{2,n-1} + x_n y_n/\sigma_{y_n}^2.$$

- The recursive approach requires an initial estimate of $c_{1,0}$ and $c_{2,0}$. One approach is to simply set $c_{1,0} = c_{2,0} = 0$.
- Alternately, we can recognize that a cell with nominal capacity Q_{nom} has that capacity over a state-of-charge range of 1.0. Therefore, we can initialize with a synthetic zeroth “measurement” where $x_0 = 1$ and $y_0 = Q_{\text{nom}}$.
- The value for $\sigma_{y_0}^2$ can be set to the manufacturing variance of the nominal capacity.
 - That is, $c_{1,0} = 1/\sigma_{y_0}^2$ and $c_{2,0} = Q_{\text{nom}}/\sigma_{y_0}^2$.
- This method may easily be adapted to allow fading memory of past measurements.
- We modify the WLS merit function to place more emphasis on recent measurements.
- We define the fading memory weighted least squares (FMWLS) merit function as

$$\chi_{\text{FMWLS}}^2 = \sum_{i=1}^N \gamma^{N-i} \frac{(y_i - \widehat{Q} x_i)^2}{\sigma_{y_i}^2},$$

where the forgetting factor γ is in the range $0 \ll \gamma \leq 1$.

- Then, the solution becomes

$$\widehat{Q} = \sum_{i=1}^N \gamma^{N-i} \frac{x_i y_i}{\sigma_{y_i}^2} \bigg/ \sum_{i=1}^N \gamma^{N-i} \frac{x_i^2}{\sigma_{y_i}^2}.$$

- This solution may also easily be computed in a recursive manner.

- We keep track of the two running sums $\tilde{c}_{1,n} = \sum_{i=1}^n \gamma^{N-i} x_i^2 / \sigma_{y_i}^2$ and

$$\tilde{c}_{2,n} = \sum_{i=1}^n \gamma^{N-i} x_i y_i / \sigma_{y_i}^2.$$

- Then, $\widehat{Q}_n = \tilde{c}_{2,n} / \tilde{c}_{1,n}$. When an additional data point becomes available, we update these quantities via

$$\tilde{c}_{1,n} = \gamma \tilde{c}_{1,n-1} + x_n^2 / \sigma_{y_n}^2$$

$$\tilde{c}_{2,n} = \gamma \tilde{c}_{2,n-1} + x_n y_n / \sigma_{y_n}^2.$$

- In summary, the WLS and FMWLS solutions have a number of nice properties:

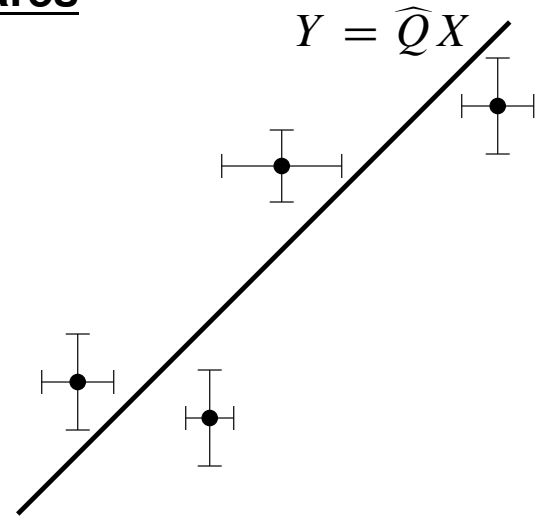
1. They give a closed-form solution for $\widehat{Q}$. Only simple operations—multiplication, addition, and division—are required.
2. The solutions can very easily be computed in a recursive manner.
3. Fading memory can easily be added, allowing adaptation of $\widehat{Q}$ to adjust for true cell total capacity changes.

4.11: Derivation of weighted total least squares

- The TLS approach assumes that there are errors on both the x_i and y_i measurements, and models the data as $(y - \Delta y) = Q(x - \Delta x)$.
- The error bars on the data point are meant to illustrate the uncertainties in each dimension, which are proportional to σ_{x_i} and σ_{y_i} .
- We assume that Δx comprises zero-mean Gaussian random variables, with known variances $\sigma_{x_i}^2$ and that Δy comprises zero-mean Gaussian random variables, with known variances $\sigma_{y_i}^2$, where $\sigma_{x_i}^2$ is not necessarily equal to or related to $\sigma_{y_i}^2$.
- TLS attempts to find an estimate $\widehat{Q}$ of the true cell total capacity Q that minimizes the sum of squared errors Δx_i plus the sum of squared errors Δy_i .
- We generalize that approach here slightly to allow for finding a $\widehat{Q}$ that minimizes the sum of *weighted* squared errors, where the weighting takes into account the uncertainty of the measurement.
- That is, we desire to find a $\widehat{Q}$ that minimizes the weighted total least squares (WTLS) merit function

$$\chi_{\text{WTLS}}^2 = \sum_{i=1}^N \frac{(x_i - X_i)^2}{\sigma_{x_i}^2} + \frac{(y_i - Y_i)^2}{\sigma_{y_i}^2}.$$

- In this equation, X_i and Y_i are the points on the line $Y_i = \widehat{Q}X_i$ corresponding to the noisy measured data pair (x_i, y_i) .



- Since both x_i and y_i have noise, we must handle this optimization problem differently from the way we handled the WLS problem.
- We augment the merit function with Lagrange multipliers λ_i to enforce the constraint that $Y_i = \widehat{Q} X_i$. This yields

$$\chi_{\text{WTLS},a}^2 = \sum_{i=1}^N \frac{(x_i - X_i)^2}{\sigma_{x_i}^2} + \frac{(y_i - Y_i)^2}{\sigma_{y_i}^2} - \lambda_i (Y_i - \widehat{Q} X_i).$$

- We set the partial derivatives of $\chi_{\text{WTLS},a}^2$ with respect to X_i , Y_i , and λ_i to zero.

$$\frac{\partial \chi_{\text{WTLS},a}^2}{\partial \lambda_i} = -(Y_i - \widehat{Q} X_i) = 0 \quad \Rightarrow \quad \boxed{Y_i = \widehat{Q} X_i}$$

$$\frac{\partial \chi_{\text{WTLS},a}^2}{\partial Y_i} = \frac{-2(y_i - Y_i)}{\sigma_{y_i}^2} - \lambda_i = 0 \quad \Rightarrow \quad \boxed{\lambda_i = \frac{-2(y_i - Y_i)}{\sigma_{y_i}^2}}$$

$$\begin{aligned} \frac{\partial \chi_{\text{WTLS},a}^2}{\partial X_i} &= \frac{-2(x_i - X_i)}{\sigma_{x_i}^2} + \lambda_i \widehat{Q} = 0 \\ 0 &= -\frac{2(x_i - X_i)}{\sigma_{x_i}^2} - \frac{2(y_i - Y_i)}{\sigma_{y_i}^2} \widehat{Q} \\ &= \sigma_{y_i}^2 (x_i - X_i) + \sigma_{x_i}^2 (y_i - Y_i) \widehat{Q} \end{aligned}$$

$$= \sigma_{y_i}^2 x_i - \sigma_{y_i}^2 X_i + \sigma_{x_i}^2 y_i \widehat{Q} - \sigma_{x_i}^2 X_i \widehat{Q}^2 \quad \Rightarrow \quad \boxed{X_i = \frac{x_i \sigma_{y_i}^2 + \widehat{Q} y_i \sigma_{x_i}^2}{\sigma_{y_i}^2 + \widehat{Q}^2 \sigma_{x_i}^2}}.$$

- With these results, we can re-write the merit function in terms of known quantities as

$$\chi_{\text{WTLS}}^2 = \sum_{i=1}^N \frac{(x_i - X_i)^2}{\sigma_{x_i}^2} + \frac{(y_i - Y_i)^2}{\sigma_{y_i}^2}$$

$$\begin{aligned}
&= \sum_{i=1}^N \frac{\left(x_i - \frac{x_i \sigma_{y_i}^2 + \widehat{Q} y_i \sigma_{x_i}^2}{\sigma_{y_i}^2 + \widehat{Q}^2 \sigma_{x_i}^2} \right)^2}{\sigma_{x_i}^2} + \frac{\left(y_i - \widehat{Q} \frac{x_i \sigma_{y_i}^2 + \widehat{Q} y_i \sigma_{x_i}^2}{\sigma_{y_i}^2 + \widehat{Q}^2 \sigma_{x_i}^2} \right)^2}{\sigma_{y_i}^2} \\
&= \sum_{i=1}^N \frac{\left(x_i \left(\sigma_{y_i}^2 + \widehat{Q}^2 \sigma_{x_i}^2 \right) - \left(x_i \sigma_{y_i}^2 + \widehat{Q} y_i \sigma_{x_i}^2 \right) \right)^2}{\sigma_{x_i}^2 \left(\sigma_{y_i}^2 + \widehat{Q}^2 \sigma_{x_i}^2 \right)^2} + \\
&\quad \frac{\left(y_i \left(\sigma_{y_i}^2 + \widehat{Q}^2 \sigma_{x_i}^2 \right) - \widehat{Q} \left(x_i \sigma_{y_i}^2 + \widehat{Q} y_i \sigma_{x_i}^2 \right) \right)^2}{\sigma_{y_i}^2 \left(\sigma_{y_i}^2 + \widehat{Q}^2 \sigma_{x_i}^2 \right)^2} \\
&= \sum_{i=1}^N \frac{\widehat{Q}^2 \sigma_{x_i}^4 (y_i - \widehat{Q} x_i)^2}{\sigma_{x_i}^2 \left(\sigma_{y_i}^2 + \widehat{Q}^2 \sigma_{x_i}^2 \right)^2} + \frac{\sigma_{y_i}^4 (y_i - \widehat{Q} x_i)^2}{\sigma_{y_i}^2 \left(\sigma_{y_i}^2 + \widehat{Q}^2 \sigma_{x_i}^2 \right)^2} \\
&= \sum_{i=1}^N \frac{(y_i - \widehat{Q} x_i)^2}{\widehat{Q}^2 \sigma_{x_i}^2 + \sigma_{y_i}^2}.
\end{aligned}$$

- To find the value of $\widehat{Q}$ that minimizes this merit function, we set the partial derivative $\partial \chi_{\text{WTLS}}^2 / \partial \widehat{Q} = 0$. That is (via Mathematica),

$$\frac{\partial \chi_{\text{WTLS}}^2}{\partial \widehat{Q}} = \sum_{i=1}^N \frac{2(\widehat{Q} x_i - y_i)(\widehat{Q} y_i \sigma_{x_i}^2 + x_i \sigma_{y_i}^2)}{(\widehat{Q}^2 \sigma_{x_i}^2 + \sigma_{y_i}^2)^2} = 0.$$

- Unfortunately, this solution has none of the nice properties of the WLS solution. Namely,
 1. There is no closed-form solution in the general case; a numerical method must be used instead to find $\widehat{Q}$.
 - One possibility is to perform a Newton–Raphson search for $\widehat{Q}$, where several iterations of the equation

$$\widehat{Q}_k = \widehat{Q}_{k-1} - \frac{\partial \chi_{\text{WTLS}}^2 / \partial \widehat{Q}}{\partial^2 \chi_{\text{WTLS}}^2 / \partial \widehat{Q}^2}$$

are performed every time the data vectors $\mathbf{x}$ and $\mathbf{y}$ are updated with new data.

- The numerator of this update equation is the “Jacobian” of the original metric function, given by the earlier equation.
- The denominator of this update equation is the “Hessian” of the original metric function, which can be found to be

$$\frac{\partial^2 \chi_{\text{WTLS}}^2}{\partial \widehat{Q}^2} = 2 \sum_{i=1}^N \left(\frac{\sigma_{y_i}^4 x_i^2 + \sigma_{x_i}^4 (3\widehat{Q}^2 y_i^2 - 2\widehat{Q}^3 x_i y_i)}{(\widehat{Q}^2 \sigma_{x_i}^2 + \sigma_{y_i}^2)^3} - \frac{\sigma_{x_i}^2 \sigma_{y_i}^2 (3\widehat{Q}^2 x_i^2 - 6\widehat{Q} x_i y_i + y_i^2)}{(\widehat{Q}^2 \sigma_{x_i}^2 + \sigma_{y_i}^2)^3} \right).$$

- The Newton–Raphson search can be initialized with a WLS estimate of $\widehat{Q}$, and has the property that the number of significant figures in the solution doubles with each iteration of the update.
 - In practice, we find that around four iterations produce double-precision results.
 - Note that the metric function χ_{WTLS}^2 is convex, so this iterative method is guaranteed to converge to the global solution.
2. There is no recursive update in the general case. This has storage implications and computational implications.
- To use WTLS, the entire vector $\mathbf{x}$ and $\mathbf{y}$ must be stored, which implies increasing storage as the number of measurements increase.
 - Furthermore, the number of computations grows as N grows.

- This is not well suited for an embedded-system application that must run in real time with limited storage capabilities.
3. There is no fading memory recursive update (because there is no recursive update).
- A *non-recursive* fading memory merit function may be defined, however, as

$$\chi_{\text{FMWTLs}}^2 = \sum_{i=1}^N \gamma^{N-i} \frac{(y_i - \widehat{Q}x_i)^2}{\widehat{Q}^2\sigma_{x_i}^2 + \sigma_{y_i}^2}.$$

- The Jacobian of this merit function is

$$\frac{\partial \chi_{\text{FMWTLs}}^2}{\partial \widehat{Q}} = 2 \sum_{i=1}^N \gamma^{N-i} \frac{(\widehat{Q}x_i - y_i)(\widehat{Q}y_i\sigma_{x_i}^2 + x_i\sigma_{y_i}^2)}{(\widehat{Q}^2\sigma_{x_i}^2 + \sigma_{y_i}^2)^2}.$$

- The Hessian is

$$\begin{aligned} \frac{\partial^2 \chi_{\text{FMWTLs}}^2}{\partial \widehat{Q}^2} = 2 \sum_{i=1}^N \gamma^{N-i} & \left(\frac{\sigma_{y_i}^4 x_i^2 + \sigma_{x_i}^4 (3\widehat{Q}^2 y_i^2 - 2\widehat{Q}^3 x_i y_i)}{(\widehat{Q}^2\sigma_{x_i}^2 + \sigma_{y_i}^2)^3} \right. \\ & \left. - \frac{\sigma_{x_i}^2 \sigma_{y_i}^2 (3\widehat{Q}^2 x_i^2 - 6\widehat{Q} x_i y_i + y_i^2)}{(\widehat{Q}^2\sigma_{x_i}^2 + \sigma_{y_i}^2)^3} \right). \end{aligned}$$

- Using the Jacobian and Hessian of this cost function, we can use a Newton–Raphson search to find the solution to the fading-memory cost function to find an estimate of Q .
- In a little while, we will address a special case of WTLS that gives a closed-form solution, with recursive update, and fading memory.
 - We will then give an approximate solution to the general WTLS problem that also has these nice properties.
 - Before we do so, we first consider two important properties of both the WLS and WTLS solutions.

4.12: Goodness of the model fit and confidence intervals

- When the measurement errors $\Delta \mathbf{x}$ and $\Delta \mathbf{y}$ are uncorrelated and Gaussian, the metric functions χ_{WLS}^2 and χ_{WTLS}^2 are chi-squared random variables.
 - χ_{WLS}^2 is a chi-squared random variable with $N - 1$ degrees of freedom, because N data points y_i were used in its creation and one degree of freedom is lost when fitting $\widehat{Q}$.
 - χ_{WTLS}^2 is a chi-squared random variable with $2N - 1$ degrees of freedom, because N data points x_i and N additional data points y_i are used in its creation, and one degree of freedom is lost when fitting $\widehat{Q}$.
- Knowledge of the distribution and the number of degrees of freedom can be used to determine, from the optimized values of the metric functions, whether the model fit is reliable; that is, whether the linear fit is a good fit to the data, and whether the optimized value of $\widehat{Q}$ is a good estimate of the cell total capacity.
- The incomplete gamma function $P(\chi^2 \mid \nu)$ is defined as the probability that the observed chi-square for a correct model should be less than a value χ^2 for degree of freedom ν .
- Its complement, $Q(\chi^2 \mid \nu) = 1 - P(\chi^2 \mid \nu)$, is the probability that the observed chi-square will exceed the value χ^2 by chance *even* for a correct model.²

² The nomenclature $Q(\chi^2 \mid \nu)$ is standard for the (complementary) incomplete gamma function, and is not to be confused with the symbol used to denote true cell total capacity Q , or with the symbol used to denote the estimate of cell total capacity $\widehat{Q}$.

- Therefore, to test for goodness of fit of a model, we must evaluate

$$Q(\chi^2 | \nu) = \frac{1}{\Gamma(\nu/2)} \int_{\chi^2/2}^{\infty} e^{-t} t^{(\nu/2-1)} dt.$$

- Methods for computing this function are built into many engineering analysis programs, and c-language code is also easy to find.
- If the value obtained for $Q(\chi^2 | \nu)$ is small, then either:
 - The model is wrong and can be statistically rejected, or
 - The variances $\sigma_{x_i}^2$ or $\sigma_{y_i}^2$ are poorly known, or
 - The variances are not actually Gaussian.
- The third possibility is common, but also generally benign if we are willing to accept low values of $Q(\chi^2 | \nu)$ as representing a valid model.
 - It is not unusual to accept models with $Q(\chi^2 | \nu) > 0.001$ and to reject them otherwise.
- We will see that when the hypothesized model is not a good fit to the data, the value of $Q(\chi^2 | \nu)$ becomes extremely small.
- However, when the hypothesized model is equal to the true model generating the data, even when $\widehat{Q}$ is not precisely equal to Q , the value of $Q(\chi^2 | \nu)$ tends to be very close to unity.
- We will use this information later to show that the WLS model is not a good approach to total capacity estimation, whereas WTLS is much better.

Evaluating the confidence limits on the estimated total capacity

- When computing an estimate of cell total capacity $\widehat{Q}$, it is also important to be able to specify the certainty of that estimate.

- Specifically, we would like to estimate the variance $\sigma_{\hat{Q}}^2$ of the total capacity estimate, with which we can compute confidence intervals such as three-sigma bounds $(\hat{Q} - 3\sigma_{\hat{Q}}, \hat{Q} + 3\sigma_{\hat{Q}})$ within which the true value of cell total capacity Q lies, with high certainty.
- To derive confidence limits, we must re-cast the least-squares type optimization problem as a maximum-likelihood optimization problem.
- With the assumption that all errors are Gaussian, this is straightforward.
- If we form a vector $\mathbf{y}$ comprising elements y_i , and a vector $\mathbf{x}$ comprising corresponding elements x_i and a diagonal matrix $\Sigma_{\mathbf{y}}$ having corresponding diagonal elements $\sigma_{y_i}^2$, then minimizing χ_{WLS}^2 is equivalent to maximizing

$$\begin{aligned} ML_{\text{WLS}} &= \frac{1}{(2\pi)^{N/2} |\Sigma_{\mathbf{y}}|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{y} - \hat{Q}\mathbf{x})^T \Sigma_{\mathbf{y}}^{-1} (\mathbf{y} - \hat{Q}\mathbf{x}) \right) \\ &= \frac{1}{(2\pi)^{N/2} |\Sigma_{\mathbf{y}}|^{1/2}} \exp \left(-\frac{1}{2} \chi_{\text{WLS}}^2 \right), \end{aligned}$$

which is a maximum likelihood problem.

- The constant to the left of the exponential causes the function to integrate to 1, yielding a valid probability density function.
- Similarly, if we form a vector $\mathbf{d}$ concatenating $\mathbf{y}$ and $\mathbf{x}$, and a vector $\hat{\mathbf{d}}$ concatenating the corresponding elements Y_i and X_i , and a diagonal matrix $\Sigma_{\mathbf{d}}$ having diagonal elements $\sigma_{y_i}^2$ followed by $\sigma_{x_i}^2$, then minimizing χ_{WTLS}^2 is equivalent to maximizing

$$ML_{\text{WTLS}} = \frac{1}{(2\pi)^N |\Sigma_{\mathbf{d}}|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{d} - \hat{\mathbf{d}})^T \Sigma_{\mathbf{d}}^{-1} (\mathbf{d} - \hat{\mathbf{d}}) \right)$$

$$= \frac{1}{(2\pi)^N |\Sigma_{\mathbf{d}}|^{1/2}} \exp \left(-\frac{1}{2} \chi_{\text{WTLS}}^2 \right).$$

- The maximum-likelihood formulation makes it possible to determine confidence intervals on $\widehat{Q}$.
- According to the Cramer–Rao theorem, a tight lower bound on the variance of $\widehat{Q}$ is given by the negative inverse of the second derivative of the argument of the exponential function, evaluated at the $\widehat{Q}$ that minimizes the least-squares cost function or maximizes the maximum-likelihood cost function.
- Therefore,

$$\sigma_{\widehat{Q}}^2 \geq 2 \left(\frac{\partial^2 \chi_{\text{WLS}}^2}{\partial \widehat{Q}^2} \right)^{-1} \quad \text{for WLS}$$

$$\sigma_{\widehat{Q}}^2 \geq 2 \left(\frac{\partial^2 \chi_{\text{WTLS}}^2}{\partial \widehat{Q}^2} \right)^{-1} \quad \text{for WTLS.}$$

- The second partial derivatives (*i.e.*, the Hessians) of the WTLS and FMWTLS metric functions have already been described in the context of a Newton–Raphson iteration.
- For WLS and FMWLS, the situation is easier. We have

$$\frac{\partial^2 \chi_{\text{WLS}}^2}{\partial \widehat{Q}^2} = 2 \sum_{i=1}^N \frac{x_i^2}{\sigma_{y_i}^2} \quad \text{and} \quad \frac{\partial^2 \chi_{\text{FMWLS}}^2}{\partial \widehat{Q}^2} = 2 \sum_{i=1}^N \gamma^{N-i} \frac{x_i^2}{\sigma_{y_i}^2},$$

which may be computed using the previously defined recursive parameters as

$$\frac{\partial^2 \chi_{\text{WLS}}^2}{\partial \widehat{Q}^2} = 2c_{1,n} \quad \text{and} \quad \frac{\partial^2 \chi_{\text{FMWLS}}^2}{\partial \widehat{Q}^2} = 2\tilde{c}_{1,n}.$$

4.13: Simplified method with proportional confidence on x_i and y_i

- The general WTLS solution provides excellent results but is impractical to implement in an embedded system.
- Therefore, we search for cases that lead to simpler implementations.
- Here, we look at an exact solution when the uncertainties on the x_i and y_i data points are proportional to each other for all i , which leads to a simple solution that can easily be implemented in an embedded system.
- With insights from this solution we will next look at an approximate WTLS solution that also has nice implementation properties.
- If $\sigma_{x_i} = k\sigma_{y_i}$, then the WTLS merit function reduces to a generalization of the standard TLS merit function.

- Substitute $\sigma_{x_i} = k\sigma_{y_i}$ into χ_{WTLS}^2 and associated results to get:

$$\chi_{\text{TLS}}^2 = \sum_{i=1}^N \frac{(x_i - X_i)^2}{k^2 \sigma_{y_i}^2} + \frac{(y_i - Y_i)^2}{\sigma_{y_i}^2} = \sum_{i=1}^N \frac{(y_i - \widehat{Q}x_i)^2}{(\widehat{Q}^2 k^2 + 1) \sigma_{y_i}^2}.$$

- Furthermore, the partial derivative of the WTLS merit function reduces to (again, via the substitution $\sigma_{x_i} = k\sigma_{y_i}$)

$$\frac{\partial \chi_{\text{TLS}}^2}{\partial \widehat{Q}} = 2 \sum_{i=1}^N \frac{(\widehat{Q}x_i - y_i)(\widehat{Q}k^2 y_i + x_i)}{(\widehat{Q}^2 k^2 + 1)^2 \sigma_{y_i}^2}.$$

- This equation may be solved for an exact solution to $\widehat{Q}$, without requiring iteration to do so.
- We first collect terms

$$\frac{\partial \chi_{\text{TLS}}^2}{\partial \widehat{Q}} = 2 \sum_{i=1}^N \frac{(\widehat{Q}x_i - y_i)(\widehat{Q}k^2 y_i + x_i)}{(\widehat{Q}^2 k^2 + 1)^2 \sigma_{y_i}^2} = 0$$

$$= \underbrace{\widehat{Q}^2 \sum_{i=1}^N k^2 \frac{x_i y_i}{\sigma_{y_i}^2}}_{a=k^2 c_{2,n}} + \underbrace{\widehat{Q} \sum_{i=1}^N \frac{x_i^2 - k^2 y_i^2}{\sigma_{y_i}^2}}_{b=c_{1,n}-k^2 c_{3,n}} + \underbrace{\sum_{i=1}^N \frac{-x_i y_i}{\sigma_{y_i}^2}}_{c=-c_{2,n}} = 0$$

$$\widehat{Q} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

- We simplify notation slightly by defining $c_{3,n} = \sum_{i=1}^n y_i^2 / \sigma_{y_i}^2$. Then,

$$\widehat{Q}_n = \frac{-(c_{1,n} - k^2 c_{3,n}) \pm \sqrt{(c_{1,n} - k^2 c_{3,n})^2 + 4k^2 c_{2,n}^2}}{2k^2 c_{2,n}}.$$

- Which of the two roots to choose? We can show that this quadratic equation always has one positive root and one negative root.
- This can be proven by forming the Routh array, and performing the Routh test on its values. The Routh array is:

$$\begin{array}{c|cc} \widehat{Q}^2 & k^2 c_{2,n} & -c_{2,n} \\ \widehat{Q}^1 & c_{1,n} - k^2 c_{3,n} & 0 \\ \widehat{Q}^0 & -c_{2,n} & 0 \end{array}$$

- The first column of the Routh array always has exactly one sign change, so there is one root of the polynomial in the right-half plane.
- The other root, therefore, must be in the left-half plane.
- By the fundamental theorem of algebra, because the coefficients $c_{1,n}$, $c_{2,n}$, and $c_{3,n}$ are real, the polynomial roots must either both be real or be complex conjugates.

- The fact that they are in different halves of the complex plane shows that they cannot be complex conjugates, and therefore must both be real.
- Therefore, we choose the larger root from the solution of the quadratic equation, which corresponds to the positive root.
- Recursive calculation is done via

$$\widehat{Q}_n = \frac{-c_{1,n} + k^2 c_{3,n} + \sqrt{(c_{1,n} - k^2 c_{3,n})^2 + 4k^2 c_{2,n}^2}}{2k^2 c_{2,n}},$$

where initialization is done by setting $x_0 = 1$ and $y_0 = Q_{\text{nom}}$.

- $\sigma_{y_i}^2$ is set to a value representing the uncertainty of the total capacity.
- Therefore, $c_{3,0} = Q_{\text{nom}}^2/\sigma_{y_i}^2$, $c_{2,0} = Q_{\text{nom}}/\sigma_{y_i}^2$ and $c_{1,0} = 1/\sigma_{y_i}^2$, and

$$c_{1,n} = c_{1,n-1} + x_n^2/\sigma_{y_i}^2$$

$$c_{2,n} = c_{2,n-1} + x_n y_n/\sigma_{y_i}^2$$

$$c_{3,n} = c_{3,n-1} + y_n^2/\sigma_{y_i}^2.$$

- The Hessian, which is required to compute the uncertainty of the estimate, may also be found in terms of the recursive parameters:

$$\begin{aligned} \frac{\partial^2 \chi_{\text{TLS}}^2}{\partial \widehat{Q}^2} &= \frac{(-4k^4 c_2) \widehat{Q}^3 + 6k^4 c_3 \widehat{Q}^2}{(\widehat{Q}^2 k^2 + 1)^3} \\ &\quad + \frac{(-6c_1 + 12c_2)k^2 \widehat{Q} + 2(c_1 - k^2 c_3)}{(\widehat{Q}^2 k^2 + 1)^3}. \end{aligned}$$

- This can be used to predict error bounds on the estimate $\widehat{Q}$. One-sigma bounds are computed as $\sqrt{2/(\partial^2 \chi_{\text{TLS}}^2/\partial \widehat{Q}^2)}$.

- Fading memory may be easily incorporated. Recursive calculation is done via

$$\widehat{Q}_n = \frac{-\tilde{c}_{1,n} + k^2\tilde{c}_{3,n} + \sqrt{(\tilde{c}_{1,n} - k^2\tilde{c}_{3,n})^2 + 4k^2\tilde{c}_{2,n}^2}}{2k^2\tilde{c}_{2,n}},$$

where initialization is done by setting $x_0 = 1$ and $y_0 = Q_{\text{nom}}$.

- $\sigma_{y_i}^2$ is set to a value representing the uncertainty of the total capacity.
- Therefore, $\tilde{c}_{3,0} = Q_{\text{nom}}^2/\sigma_{y_i}^2$, $\tilde{c}_{2,0} = Q_{\text{nom}}/\sigma_{y_i}^2$ and $\tilde{c}_{1,0} = 1/\sigma_{y_i}^2$, and

$$\tilde{c}_{1,n} = \gamma\tilde{c}_{1,n-1} + x_n^2/\sigma_{y_i}^2$$

$$\tilde{c}_{2,n} = \gamma\tilde{c}_{2,n-1} + x_n y_n / \sigma_{y_i}^2$$

$$\tilde{c}_{3,n} = \gamma\tilde{c}_{3,n-1} + y_n^2/\sigma_{y_i}^2.$$

- After some straightforward manipulations, we can obtain the Hessian in terms of the recursive parameters $\tilde{c}_1$ through $\tilde{c}_3$:

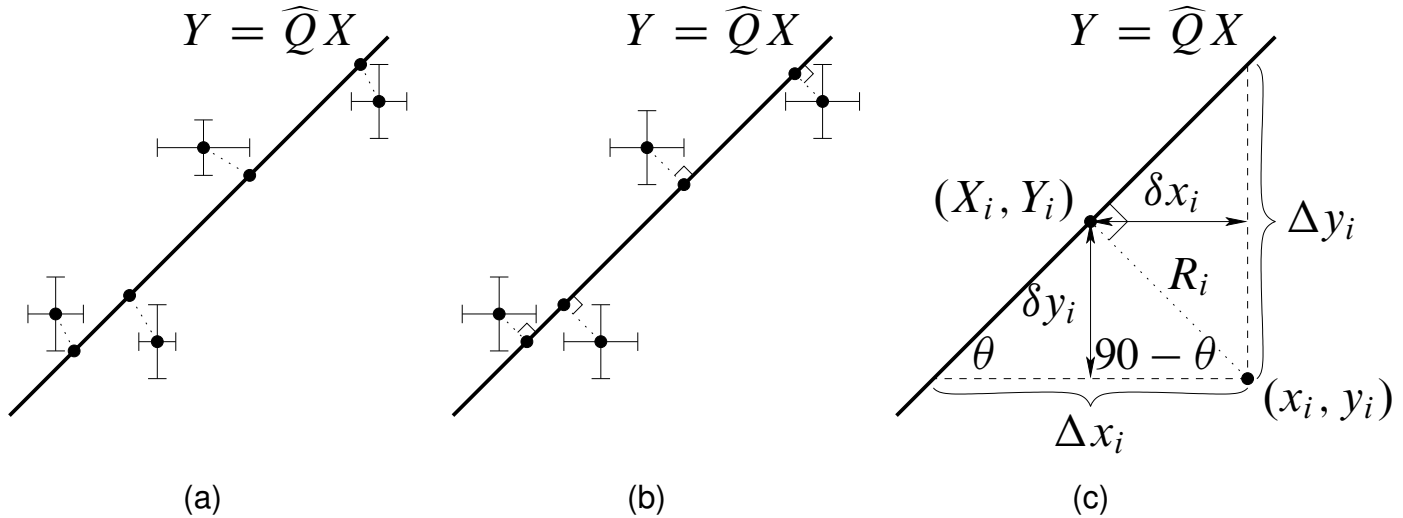
$$\begin{aligned} \frac{\partial^2 \chi_{\text{FMTLS}}^2}{\partial \widehat{Q}^2} = & \frac{(-4k^4\tilde{c}_2)\widehat{Q}^3 + 6k^4\tilde{c}_3\widehat{Q}^2}{(\widehat{Q}^2k^2 + 1)^3} \\ & + \frac{(-6\tilde{c}_1 + 12\tilde{c}_2)k^2\widehat{Q} + 2(\tilde{c}_1 - k^2\tilde{c}_3)}{(\widehat{Q}^2k^2 + 1)^3}. \end{aligned}$$

- So, this TLS solution shares the nice properties of the WLS solution:
 1. It gives a closed-form solution for $\widehat{Q}$. No iteration or advanced algorithms are required—only simple mathematical operations.
 2. The solution can be very easily computed in a recursive manner. We keep track of the three running sums $c_{1,n}$, $c_{2,n}$ and $c_{3,n}$. When an additional data point becomes available, we update the sums and compute an updated total capacity estimate.
 3. Fading memory is easily added.

- Unfortunately, this solution does not allow $\sigma_{x_i}^2$ and $\sigma_{y_i}^2$ to be arbitrary—they must be proportionally related by the scaling factor $\sigma_{x_i} = k\sigma_{y_i}$.
- The next section describes an approximation to TLS that allows an arbitrary relationship.

4.14: Approximate full solution: Cost function

- We desire an approximate solution to the WTLS problem that allows $\sigma_{x_i}^2$ and $\sigma_{y_i}^2$ to be non-proportional, but which yields a recursive solution for feasible implementation in an embedded system.



- The figure shows the geometry of WTLS and motivates the approximate solution to be developed in this section.
- Frame (a) shows relationship between data point (x_i, y_i) and its optimized map (X_i, Y_i) on $Y_i = \widehat{Q}X_i$ when $\sigma_{x_i}^2$ and $\sigma_{y_i}^2$ are arbitrary.
 - The error bars on each data point illustrate the uncertainties in each dimension, which are proportional to σ_{x_i} and σ_{y_i} .
- We see that the distance between x_i and X_i is not necessarily equal to the distance between y_i and Y_i .
- If the quality of the x_i measurement is better (poorer) than the quality of the y_i measurement, the distance to its map X_i should be shorter (greater) than the distance from y_i to its map Y_i .
- Frame (b) shows relationship between data point (x_i, y_i) and its optimized map (X_i, Y_i) on $Y_i = \widehat{Q}X_i$ when $\sigma_{x_i}^2$ and $\sigma_{y_i}^2$ are equal.

- In this case, the distance between x_i and X_i is equal to the distance between y_i and Y_i , and the line joining data point (x_i, y_i) and (X_i, Y_i) is perpendicular to the line $Y_i = \widehat{Q}X_i$.
 - If σ_{x_i} and σ_{y_i} are not equal but proportional, the x - or y -axis may be scaled to yield transformed data points with equal variances, and hence the same idea applies.
- Frame (c) illustrates definitions that will be used to derive an approximate weighted total least squares (AWTLS) solution.
- As with the TLS solution, we enforce that the line joining data point (x_i, y_i) and (X_i, Y_i) be perpendicular to the line $Y_i = \widehat{Q}X_i$.
- This will result in a solution that may be solved recursively.
- However, as with the WTLS solution, we weight the distance between x_i and X_i differently from the distance between y_i and Y_i in the optimization merit function.
- This will give a better total capacity estimate than TLS when the uncertainties on x_i and y_i are not proportional.
- We define Δx_i be the x -distance between data point i and the line, and Δy_i be the y -distance between data point i and the line.
- The slope of the line is $\widehat{Q} = \Delta y_i / \Delta x_i$ for all i .
- The angle of the line is $\theta = \tan^{-1} \widehat{Q}$.
- The shortest distance between the line and a given data point is

$$R_i = \Delta y_i \cos \theta = \Delta y_i / \sqrt{1 + \tan^2 \theta} = \Delta y_i / \sqrt{1 + \widehat{Q}^2}.$$

- Let $\delta x_i = R_i \sin \theta$ and $\delta y_i = R_i \cos \theta$.

- These are the x - and y -components of the perpendicular distance between data point i and the fitting line.
- We then weigh our fitting merit function according to these variances.
- Therefore, we define the approximate weighted total least squares (AWTLS) merit function as

$$\chi_{\text{AWTLS}}^2 = \sum_{i=1}^N \frac{\delta x_i^2}{\sigma_{x_i}^2} + \frac{\delta y_i^2}{\sigma_{y_i}^2}.$$

- Note that $\sin^2 \theta = 1 - \cos^2 \theta = \widehat{Q}^2 / (1 + \widehat{Q}^2)$. Therefore,

$$\begin{aligned} \delta x_i^2 &= \left(\frac{\Delta y_i^2}{1 + \widehat{Q}^2} \right) \left(\frac{\widehat{Q}^2}{1 + \widehat{Q}^2} \right) \\ \delta y_i^2 &= \left(\frac{\Delta y_i^2}{1 + \widehat{Q}^2} \right) \left(\frac{1}{1 + \widehat{Q}^2} \right). \end{aligned}$$

- Therefore, since $\Delta y_i = y_i - \widehat{Q}x_i$,

$$\chi_{\text{AWTLS}}^2 = \sum_{i=1}^N \frac{(y_i - \widehat{Q}x_i)^2}{(1 + \widehat{Q}^2)^2} \left(\frac{\widehat{Q}^2}{\sigma_{x_i}^2} + \frac{1}{\sigma_{y_i}^2} \right).$$

- To verify that AWTLS is an approximation to WTLS in at least some cases, we note that the two merit functions are equal when $\sigma_{x_i} = \sigma_{y_i}$.
- However, they are not equal when $\sigma_{x_i} = k\sigma_{y_i}$, but this will be corrected in the next section.

4.15: Approximate full solution: Derivation

- The Jacobian of the AWTLS merit function is

$$\begin{aligned} \frac{\partial \chi_{\text{AWTLS}}^2}{\partial \widehat{Q}} = & \frac{2}{(\widehat{Q}^2 + 1)^3} \sum_{i=1}^N \widehat{Q}^4 \left(\frac{x_i y_i}{\sigma_{x_i}^2} \right) + \widehat{Q}^3 \left(\frac{2x_i^2}{\sigma_{x_i}^2} - \frac{x_i^2}{\sigma_{y_i}^2} - \frac{y_i^2}{\sigma_{x_i}^2} \right) \\ & + \widehat{Q}^2 \left(\frac{3x_i y_i}{\sigma_{y_i}^2} - \frac{3x_i y_i}{\sigma_{x_i}^2} \right) + \widehat{Q} \left(\frac{x_i^2 - 2y_i^2}{\sigma_{y_i}^2} + \frac{y_i^2}{\sigma_{x_i}^2} \right) + \left(\frac{-x_i y_i}{\sigma_{y_i}^2} \right). \end{aligned}$$

- This can be re-written in terms of recursively computed running summations

$$\begin{aligned} \frac{\partial \chi_{\text{AWTLS}}^2}{\partial \widehat{Q}} = & \frac{2}{(\widehat{Q}^2 + 1)^3} \left(c_5 \widehat{Q}^4 + (2c_4 - c_1 - c_6) \widehat{Q}^3 \right. \\ & \left. + (3c_2 - 3c_5) \widehat{Q}^2 + (c_1 - 2c_3 + c_6) \widehat{Q} - c_2 \right). \end{aligned}$$

where initialization is done by setting $x_0 = 1$ and $y_0 = Q_{\text{nom}}$.

- $\sigma_{y_0}^2$ is set to a representative value of the uncertainty of the total capacity and $\sigma_{x_0}^2$ is set to a representative value of the uncertainty of a difference between two SOC estimates.
- Therefore, $c_{1,0} = 1/\sigma_{y_0}^2$, $c_{2,0} = Q_{\text{nom}}/\sigma_{y_0}^2$, $c_{3,0} = Q_{\text{nom}}^2/\sigma_{y_0}^2$, $c_{4,0} = 1/\sigma_{x_0}^2$, $c_{5,0} = Q_{\text{nom}}/\sigma_{x_0}^2$, $c_{6,0} = Q_{\text{nom}}^2/\sigma_{x_0}^2$, and

$$c_{1,n} = c_{1,n-1} + x_n^2/\sigma_{y_n}^2$$

$$c_{4,n} = c_{4,n-1} + x_n^2/\sigma_{x_n}^2$$

$$c_{2,n} = c_{2,n-1} + x_n y_n/\sigma_{y_n}^2$$

$$c_{5,n} = c_{5,n-1} + x_n y_n/\sigma_{x_n}^2$$

$$c_{3,n} = c_{3,n-1} + y_n^2/\sigma_{y_n}^2$$

$$c_{6,n} = c_{6,n-1} + y_n^2/\sigma_{x_n}^2.$$

- The roots of this quartic equation

$$c_5 \widehat{Q}^4 + (2c_4 - c_1 - c_6) \widehat{Q}^3 + (3c_2 - 3c_5) \widehat{Q}^2 + (c_1 - 2c_3 + c_6) \widehat{Q} - c_2 = 0$$

are candidate solutions for $\widehat{Q}$.

- They may be found using the Ferrari method, a closed-form solution that does not require iterative or eigenvalue analysis, so may be implemented readily in an embedded system.
- However, of the four roots only one is the optimum, and we do not know any method to decide *a priori* which root to solve for.
- In our experience, with some sets of data all roots are real, but with other sets of data some can be complex, and some can be negative.
- The only foolproof method we know to determine which root is the solution that minimizes χ_{AWTLS}^2 is to evaluate the merit function χ_{AWTLS}^2 at each of the four candidate solutions, and to retain the one that gives the lowest value. (This can be skipped for negative and complex roots, which are not possible solutions for battery cell capacity.)
- Computing the merit function may be very readily done if we rewrite it in terms of the running summations

$$\chi_{\text{AWTLS}}^2 = \frac{1}{(\widehat{Q}^2 + 1)^2} (c_4 \widehat{Q}^4 - 2c_5 \widehat{Q}^3 + (c_1 + c_6) \widehat{Q}^2 - 2c_2 \widehat{Q} + c_3).$$

- When the assumptions made in deriving AWTLS are approximately true, the Hessian yields a good value for the error bounds on the total capacity estimate.
- After some straightforward but messy mathematics, we can find the Hessian to be

$$\begin{aligned} \frac{\partial^2 \chi_{\text{AWTLS}}^2}{\partial \widehat{Q}^2} = \frac{2}{(\widehat{Q}^2 + 1)^4} & \left(-2c_5 \widehat{Q}^5 + (3c_1 - 6c_4 + 3c_6) \widehat{Q}^4 \right. \\ & + (-12c_2 + 16c_5) \widehat{Q}^3 + (-8c_1 + 10c_3 + 6c_4 - 8c_6) \widehat{Q}^2 \\ & \left. + (12c_2 - 6c_5) \widehat{Q} + (c_1 - 2c_3 + c_6) \right). \end{aligned}$$

- Fading memory can be easily incorporated. The cost function is

$$\chi_{\text{FMAWTLS}}^2 = \sum_{i=1}^N \gamma^{N-i} \frac{(y_i - \widehat{Q} x_i)^2}{(1 + \widehat{Q}^2)^2} \left(\frac{\widehat{Q}^2}{\sigma_{x_i}^2} + \frac{1}{\sigma_{y_i}^2} \right).$$

- The Jacobian is

$$\begin{aligned} \frac{\partial \chi_{\text{AWTLS}}^2}{\partial \widehat{Q}} = \frac{2}{(\widehat{Q}^2 + 1)^3} \sum_{i=1}^N \gamma^{N-i} \left[\widehat{Q}^4 \left(\frac{x_i y_i}{\sigma_{x_i}^2} \right) + \widehat{Q}^3 \left(\frac{2x_i^2}{\sigma_{x_i}^2} - \frac{x_i^2}{\sigma_{y_i}^2} - \frac{y_i^2}{\sigma_{x_i}^2} \right) \right. \\ \left. + \widehat{Q}^2 \left(\frac{3x_i y_i}{\sigma_{y_i}^2} - \frac{3x_i y_i}{\sigma_{x_i}^2} \right) + \widehat{Q} \left(\frac{x_i^2 - 2y_i^2}{\sigma_{y_i}^2} + \frac{y_i^2}{\sigma_{x_i}^2} \right) + \left(\frac{-x_i y_i}{\sigma_{y_i}^2} \right) \right]. \end{aligned}$$

- This can be re-written in terms of recursively computed running summations

$$\begin{aligned} \frac{\partial \chi_{\text{FMAWTLS}}^2}{\partial \widehat{Q}} = \frac{2}{(\widehat{Q}^2 + 1)^3} \left(\tilde{c}_5 \widehat{Q}^4 + (-\tilde{c}_1 + 2\tilde{c}_4 - \tilde{c}_6) \widehat{Q}^3 \right. \\ \left. + (3\tilde{c}_2 - 3\tilde{c}_5) \widehat{Q}^2 + (\tilde{c}_1 - 2\tilde{c}_3 + \tilde{c}_6) \widehat{Q} - \tilde{c}_2 \right). \end{aligned}$$

where initialization is done by setting $x_0 = 1$ and $y_0 = Q_{\text{nom}}$.

- $\sigma_{y_0}^2$ is set to a representative value of the uncertainty of the total capacity and $\sigma_{x_0}^2$ is set to a representative value of the uncertainty of a difference between two SOC estimates.
- Therefore, $\tilde{c}_{1,0} = 1/\sigma_{y_0}^2$, $\tilde{c}_{2,0} = Q_{\text{nom}}/\sigma_{y_0}^2$, $\tilde{c}_{3,0} = Q_{\text{nom}}^2/\sigma_{y_0}^2$, $\tilde{c}_{4,0} = 1/\sigma_{x_0}^2$, $\tilde{c}_{5,0} = Q_{\text{nom}}/\sigma_{x_0}^2$, $\tilde{c}_{6,0} = Q_{\text{nom}}^2/\sigma_{x_0}^2$, and

$$\tilde{c}_{1,n} = \gamma \tilde{c}_{1,n-1} + x_n^2/\sigma_{y_n}^2$$

$$\tilde{c}_{4,n} = \gamma \tilde{c}_{4,n-1} + x_n^2/\sigma_{x_n}^2$$

$$\tilde{c}_{2,n} = \gamma \tilde{c}_{2,n-1} + x_n y_n/\sigma_{y_n}^2$$

$$\tilde{c}_{5,n} = \gamma \tilde{c}_{5,n-1} + x_n y_n/\sigma_{x_n}^2$$

$$\tilde{c}_{3,n} = \gamma \tilde{c}_{3,n-1} + y_n^2/\sigma_{y_n}^2$$

$$\tilde{c}_{6,n} = \gamma \tilde{c}_{6,n-1} + y_n^2/\sigma_{x_n}^2.$$

- Again, this quartic equation may be solved using the Ferrari method, for example. The four candidate solutions must be checked against the merit function to determine which one is optimal.

- The merit function in terms of these variables is

$$\chi_{\text{FMAWTLS}}^2 = \frac{1}{(\widehat{Q}^2 + 1)^2} (\tilde{c}_4 \widehat{Q}^4 - 2\tilde{c}_5 \widehat{Q}^3 + (\tilde{c}_1 + \tilde{c}_6) \widehat{Q}^2 - 2\tilde{c}_2 \widehat{Q} + \tilde{c}_3).$$

- The Hessian is

$$\begin{aligned} \frac{\partial^2 \chi_{\text{FMAWTLS}}^2}{\partial \widehat{Q}^2} = \frac{2}{(\widehat{Q}^2 + 1)^4} & \left(-2\tilde{c}_5 \widehat{Q}^5 + (3\tilde{c}_1 - 6\tilde{c}_4 + 3\tilde{c}_6) \widehat{Q}^4 \right. \\ & + (-12\tilde{c}_2 + 16\tilde{c}_5) \widehat{Q}^3 + (-8\tilde{c}_1 + 10\tilde{c}_3 + 6\tilde{c}_4 - 8\tilde{c}_6) \widehat{Q}^2 \\ & \left. + (12\tilde{c}_2 - 6\tilde{c}_5) \widehat{Q} + (\tilde{c}_1 - 2\tilde{c}_3 + \tilde{c}_6) \right). \end{aligned}$$

- Note that the AWTLS merit function does not equal the WTLS merit function when $\sigma_{y_i} = k\sigma_{x_i}$.
- This can be easily remedied. Define $\tilde{y}_i = ky_i$. Then $\sigma_{\tilde{y}_i} = \sigma_{x_i}$.
- Invoke the AWTLS or FMAWTLS methods to find total capacity estimate $\widehat{Q}$ and Hessian H using input sequences comprised of the original $\mathbf{x}$ vector and the scaled $\tilde{\mathbf{y}}$ vector (i.e., $(x_i, \tilde{y}_i)$ with corresponding variances $(\sigma_{x_i}^2, k^2\sigma_{y_i}^2)$).
- The true slope estimate can be found as $\widehat{Q}_{\text{corrected}} = \widehat{Q}/k$ and the corrected Hessian can be found as $H_{\text{corrected}} = H/k^2$.
- This is the method used in the results section, where the proportionality constant is estimated as $k = \sigma_{x_1}/\sigma_{y_1}$.
- This scaling improves results even when σ_{y_i} and σ_{x_i} are not proportionally related, if k is chosen to give an “order of magnitude”

proportionality or average proportionality between the uncertainties of x_i and y_i .

- In summary, these AWTLS solutions share the nice properties of the WLS solution:
 1. They give a closed-form solution for $\widehat{Q}$. No iteration or advanced algorithms are required—only simple mathematical operations.
 2. The solution can be very easily computed in a recursive manner. We keep track of the six running sums $c_{1,n}$ through $c_{6,n}$. When an additional data point becomes available, we update the sums and compute an updated total capacity estimate.
 3. Fading memory can easily be added to allow the estimate $\widehat{Q}$ to place greater emphasis on more recent measurements than on earlier measurements, allowing adaptation of $\widehat{Q}$ to adjust for true cell total capacity changes.
 4. Furthermore, this method is superior to the TLS solution since it allows individual weighting on the x_i and y_i data points.

4.16: Example simulations, HEV cases

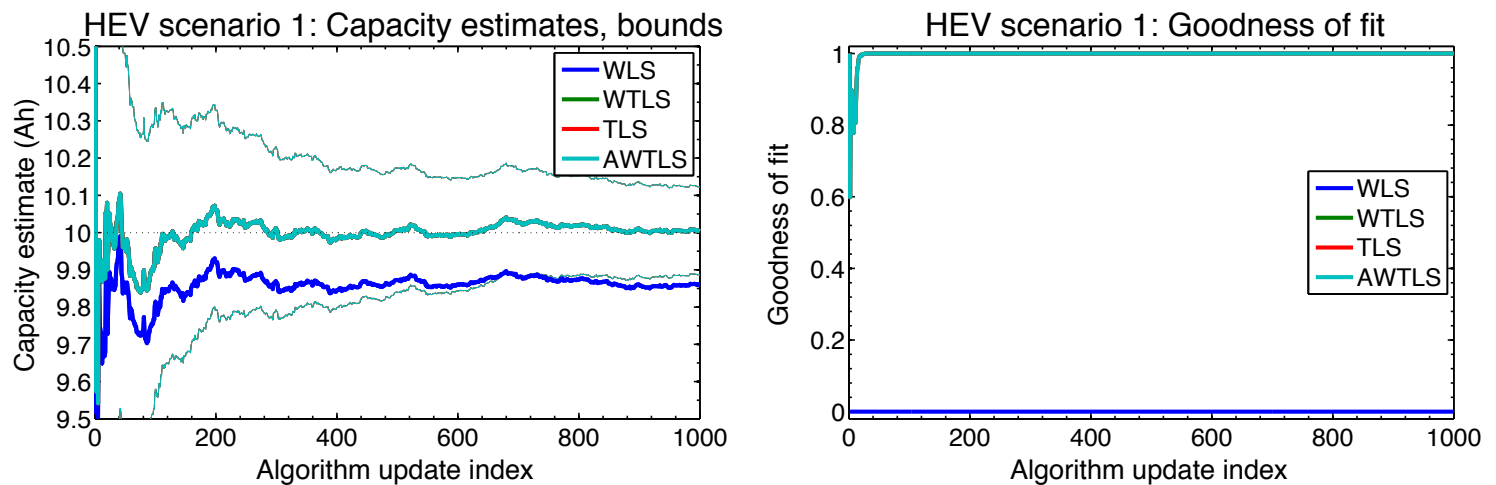
- This section presents a number of usage scenarios to exercise the total capacity estimation methods and compare their performance.
- All scenarios use the fading-memory version of the four methods, but we omit the prefix “FM” for brevity.
- Unless otherwise stated, the fading memory forgetting factor $\gamma = 1.0$.
- We assume that the individual SOC estimates that are input to these methods can be determined to an accuracy of $\sigma_z = 0.01$.
- This is being very generous, since the best method we are aware of, SPKF, achieves only around $\sigma_z = 0.01$ for LMO cells and $\sigma_z = 0.03$ for LFP cells in practice, when Q_{nom} is used instead of Q in the estimator.
- Other methods we have used, such as EKF, achieve around $\sigma_z = 0.02$ or higher for LMO cells in practice.
 - A nice advantage of both EKF and SPKF is that they give dynamic estimates of σ_z that ensure that the values of σ_{x_i} used in total capacity estimation are accurate.
- However, despite limitations in present SOC estimation, we are confident that with better cell modeling, these methods can be improved in the future.
- We use computer simulation rather than cell testing to validate the algorithms because it allows us to constrain a variety of factors that would be difficult to control in a real-time embedded system.
- These include:

- The efficacy and accuracy of the SOC estimation algorithms used to provide input to the total capacity estimation algorithms;
 - The accuracy and precision of the raw sensor measurements used as input (including the challenges of bias errors, nonlinear errors, and random errors, for example);
 - The repeatability of the experiment; and
 - The fact that total capacity of a physical cell fades over time and the associated difficulty/impossibility of knowing the “true” value of total capacity with which to compare our results.
- Therefore, we choose to use synthetic data to isolate the performance of the total capacity estimation algorithms themselves, when all other factors are in some sense idealized.
 - The x_i and y_i values are mathematically generated, as described in the individual subsections below.

Hybrid electric vehicle application, scenario 1

- The first sets of simulations that we present are for hybrid electric vehicle scenarios.
- From the perspective of total capacity estimation, these applications are characterized by the narrow window of SOC used.
 - We assume that the vehicle uses a SOC range of 40 % to 60 %.
 - Therefore, each time the total capacity estimate is updated, the true change in SOC can range from -0.2 to $+0.2$.
 - We simulate this by choosing the true value of x_i to be a uniform random number selected between these limits.

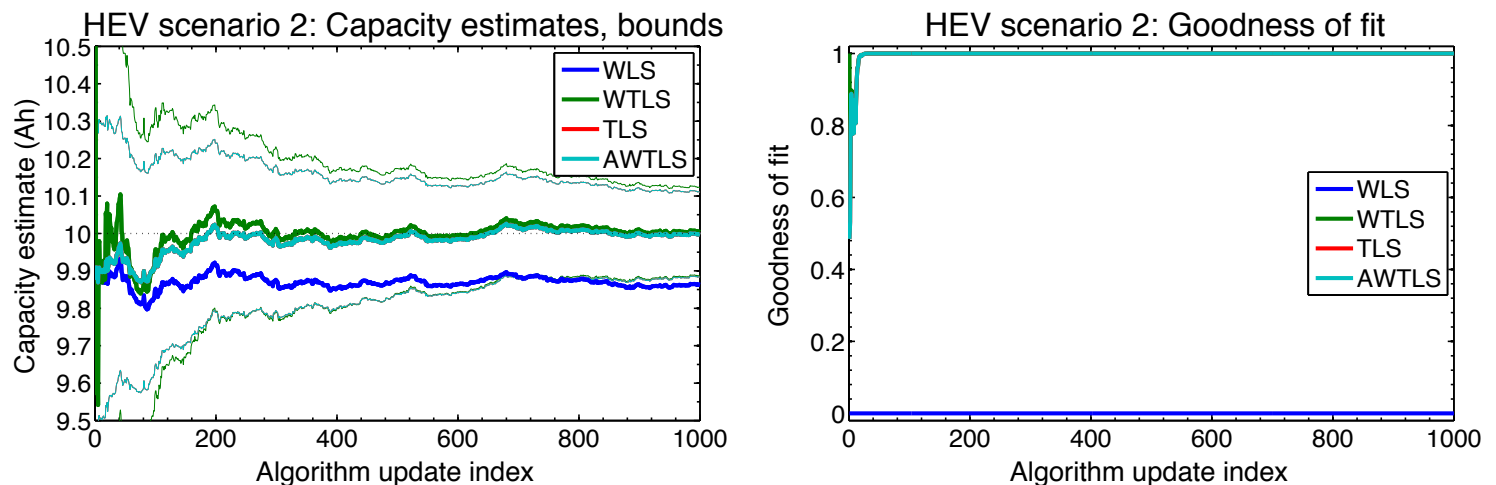
- The HEV application is also characterized by the fact that the battery pack is never fully charged to a precisely known SOC; therefore, each time the total capacity estimate is updated, two estimates of SOC are required to compute $x = z(t_2) - z(t_1)$.
- This gives an overall $\sigma_x^2 = 2\sigma_z^2 = 2(0.01)^2$.
- We simulate this by computing the “measured” value of x_i to be equal to the true value of x_i added to a zero-mean Gaussian random number having variance σ_x^2 .
- We compute the true value of y_i to be equal to the nominal capacity of the cell Q_{nom} multiplied by the true value of x_i .
- Noise on the y_i measurement is assumed to comprise accumulated quantization noises.
- For y_i computed by summing m_i measurements, taken at a 1 Hz rate, from a sensor having quantizer resolution q , the total noise is $\sigma_{y_i}^2 = q^2 m_i / (12 \times 3600^2)$.
- For HEV scenario 1, we assumed that the maximum range of the current sensor is $\pm 30 Q_{\text{nom}}$ and that a 10 bit A2D is used to measure current.
- This leads to $q = 60 Q_{\text{nom}} / 1024$.
- We chose $m_i = 300$ s for every measurement and a nominal capacity of $Q_{\text{nom}} = 10$ Ah.
- The recursive estimates were not initialized prior to the first data point being received (equivalently, the recursive parameters were initialized to zero). Results are presented below:



- The left frame shows estimates made using the four methods evolving over time as thick lines, and their three-sigma error bounds, computed using the Hessian method, as thin lines.
- We see that WTLS, TLS, and AWTLS give identical estimates and error bounds under this scenario, and converge to the neighborhood of the true total capacity.
- The WLS estimate is biased, and the error bounds are (incorrectly) so tight that they are indistinguishable from the estimate itself.
- The right frame shows the goodness of fit metric as applied to the four methods.
- Again, WTLS, TLS, and AWTLS give identical results, quickly converging to a value of 1.0 (i.e., the methods are confident that their estimate is reliable).
- The WLS method returns a vanishingly small value for goodness of fit, reflecting the fact that the method does not give a good value for its total capacity estimate and/or bounds.

Hybrid electric vehicle application, scenario 2

- The second HEV scenario is identical to the first, except that the recursive methods are initialized with a total capacity estimate before further measurements are received.
- In this case, the methods were initialized with a “nominal” capacity estimate of 9.9 Ah (the true total capacity was still 10.0). Results are presented below:

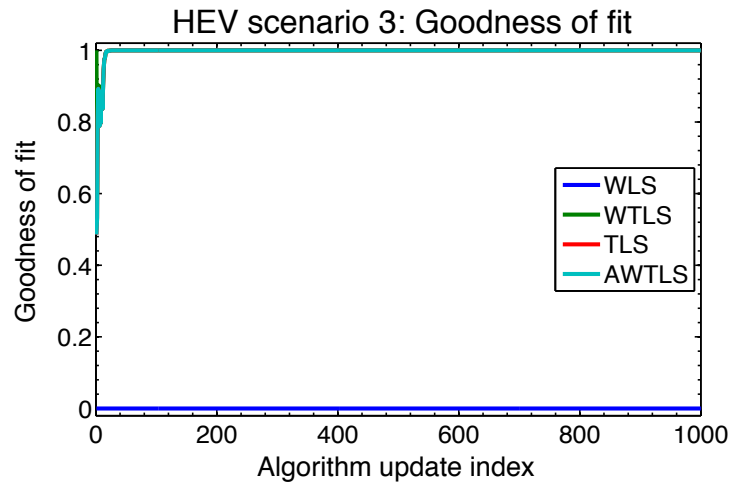
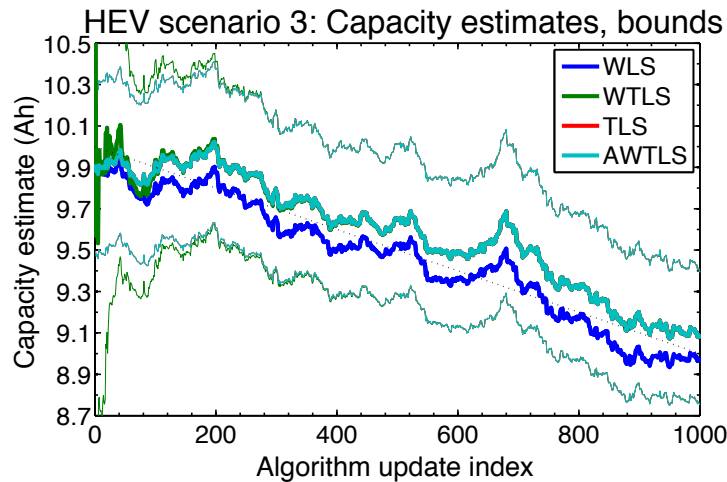


- In this scenario, TLS and AWTLS give identical results for both their estimates, error bounds, and goodness of fit.
- WTLS cannot be calculated recursively, so the estimation cannot be initialized—its results are the same as for scenario 1.
- Once again, WLS is inferior to the other methods.
- TLS and AWTLS give the best results because of tighter error bounds while maintaining a high value for goodness of fit.

Hybrid electric vehicle application, scenario 3

- In the third HEV scenario, we explore the ability of the algorithms to track a moving value of total capacity.

- This scenario is identical to HEV scenario 2, except that the true total capacity is changing with a slope of -0.001 Ah per measurement update, and a fading memory forgetting factor of $\gamma = 0.99$ is used for all methods.
- Results are presented below, where the true total capacity is drawn as a dotted black line.

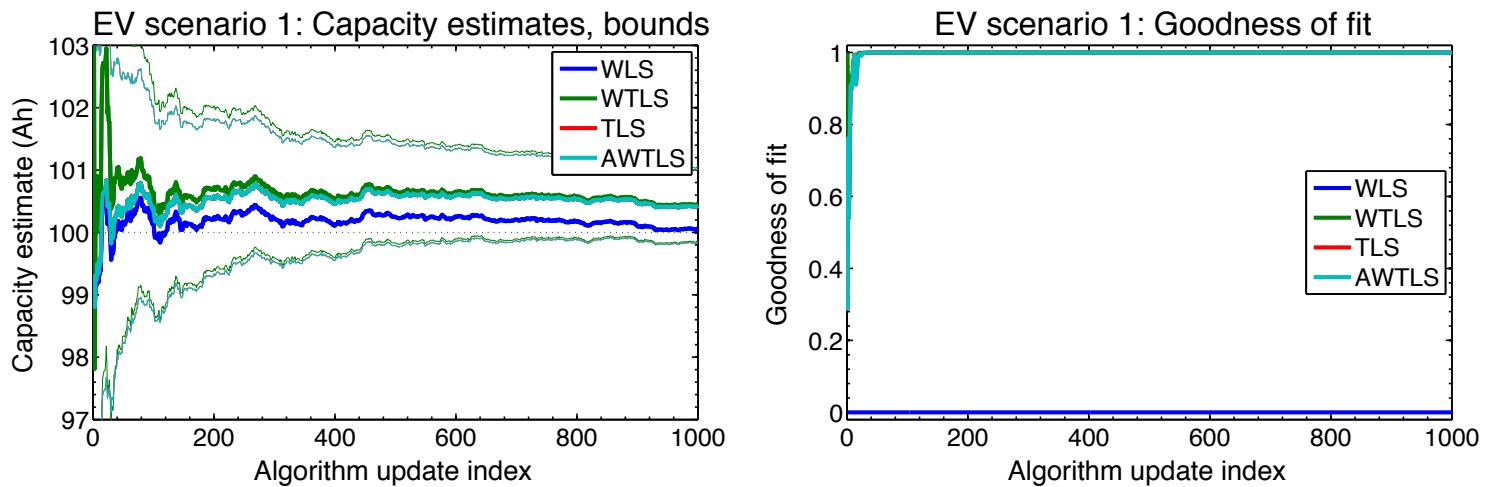


- In this example, the WLS method *appears* to give good results, but its goodness of fit value is still vanishingly small because the error bounds are unreasonably tight, and almost never surround the true value of total capacity.
- WTLS, TLS, and AWTLS are also able to track the moving value of total capacity—TLS and AWTLS give the best results due to the ability to initialize with a reasonable initial value, yielding narrower error bounds.

4.17: Example simulations, BEV cases

Battery electric vehicle application, scenario 1

- The next scenarios that we consider are typical of battery electric vehicle and plug-in hybrid electric vehicle operation.
- These are different from HEV application in several respects: the battery total capacity is larger, the relative rate of energy usage is lower, the range of SOC used by the vehicle is larger, and the BEV battery pack is sometimes fully charged to a known set point.
- In all cases, we consider a battery pack with total capacity of $Q_{\text{nom}} = 100 \text{ Ah}$, and a maximum rate of $\pm 5Q_{\text{nom}}$.
- We again assume a 10-bit current sensor, which gives $q = 10Q_{\text{nom}}/1024$ and $\sigma_{y_i}^2 = q^2 m_i / (12 \times 3600^2)$.
- For the first BEV scenario we assume that the total capacity estimate is updated on a regular basis as the vehicle operates, with $m_i = 7200 \text{ s}$ (*i.e.*, 2 h or about 120 mi).
- We assume that the battery SOC can change by $\pm 40\%$ in that interval, so the true value of x_i is chosen to be a uniform random variable between -0.4 and $+0.4$.
- Again, noise on x_i is Gaussian with variance $\sigma_{x_i}^2 = 2(0.01)^2$. The recursive methods are initialized with an initial total capacity estimate of 99 Ah.
- Representative results of this scenario are presented below.

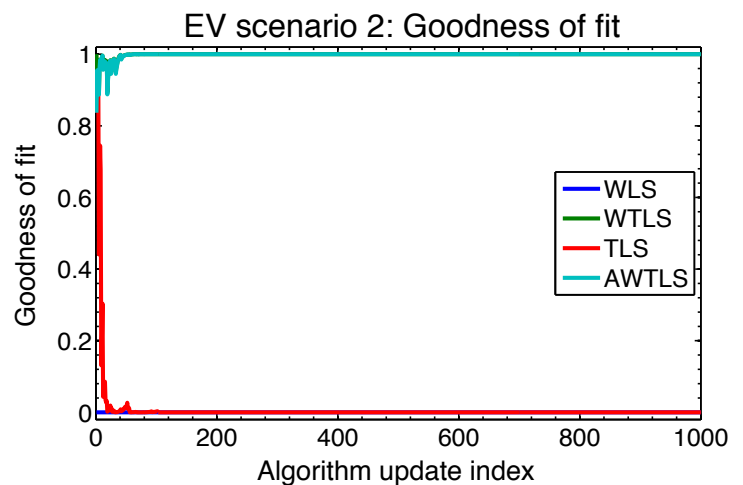
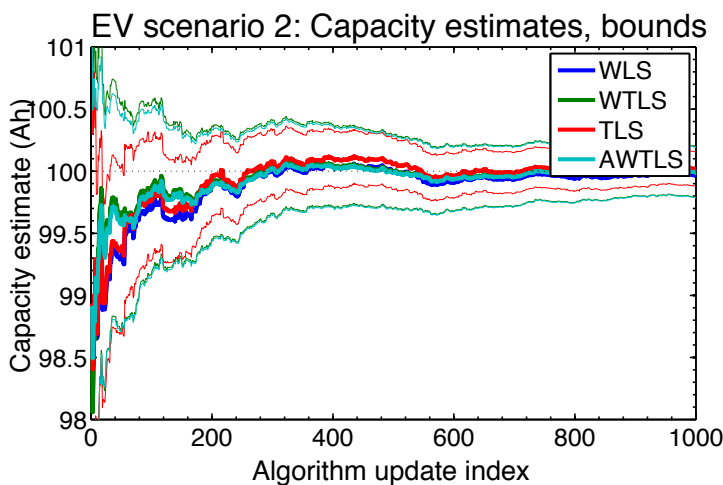
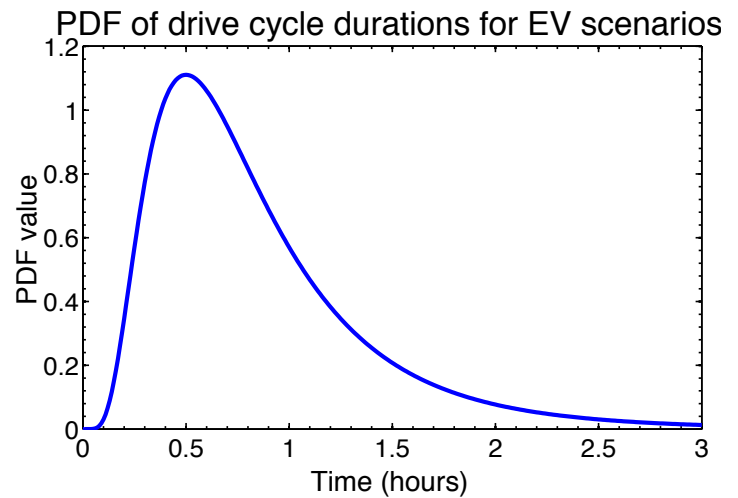


- These are very similar in most respects to the HEV scenario 2 results.
- Again, WLS fails because its error bounds are far too tight, leading to a vanishingly small goodness of fit.
- WTLS, TLS, and AWTLS all give good results, with TLS and AWTLS giving the best results due to lower error bounds because of the possibility of initialization.

Battery electric vehicle application, scenario 2

- The asymptotic quality of the total capacity estimates is limited by the noise on the SOC estimation error.
- If this noise can be reduced, the total capacity estimates can become much more accurate.
- The BEV application allows a means to do this: whenever the battery pack is fully charged, we have a precisely known end-point SOC.
- Therefore, either $z(t_1)$ or $z(t_2)$ can be known “exactly” for every total capacity estimate update.
- This then allows us to use $\sigma_{x_i}^2 = \sigma_z^2 = (0.01)^2$.

- The tradeoff is that we no longer have regular updates.
- Updates happen randomly, whenever the vehicle is charged.
- Therefore, m_i becomes a random variable.
- For this work, we treat m_i as a log-normal random variable with mode 0.5 hours and standard deviation 0.6 hours.
- Other PDFs could easily be used: this one was chosen to give reasonable duration drive cycles that encompassed a variety of driving behaviors and distances.
- Also, since a greater fraction of the battery pack would be used for an entire drive cycle than for a regular periodic update, we use an 80 % range of SOC, so the true value of x_i is computed to be a uniform random number from -0.8 to $+0.8$.
- Results from this scenario are presented below.

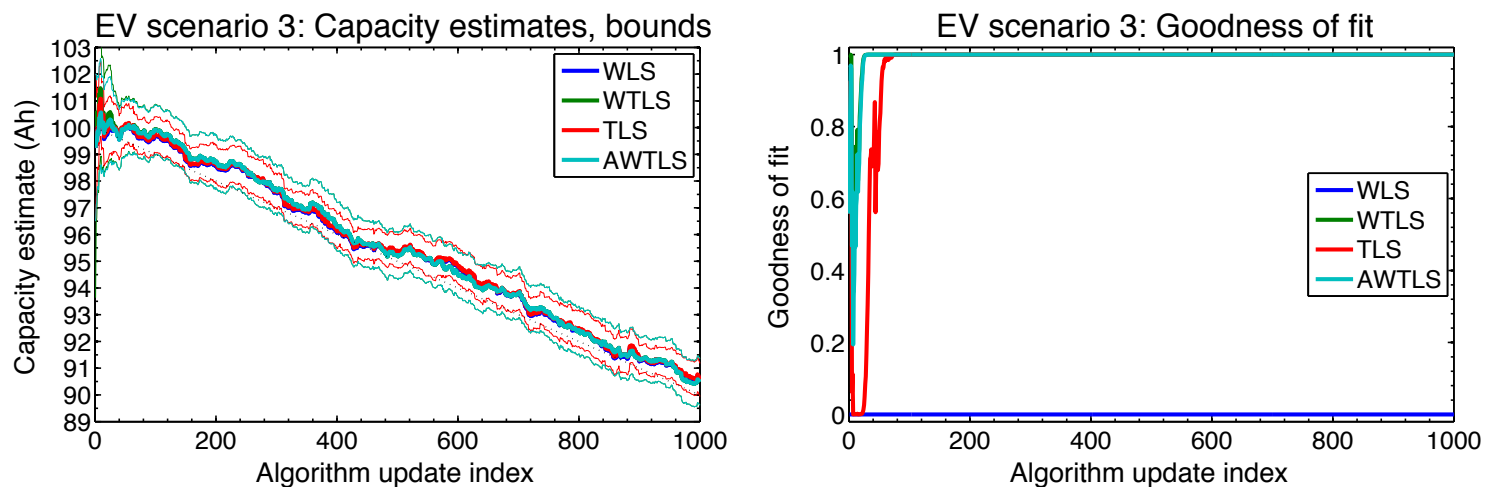


- WLS fails once again, but this time TLS also fails. TLS fails because $\sigma_{x_i} \neq k\sigma_{y_i}$ due to the variable-length drive cycles.

- The estimate given by TLS is actually quite reasonable, but the goodness of fit is very small.
- WTLS gives good results, and AWTLS gives the best results due to lower error bounds because of the ability to initialize the estimate.
- Note that the asymptotic three-sigma error bounds drop from about $\pm 1\%$ to about $\pm 0.15\%$ of the total capacity due to having a lower value of σ_{x_i} and also due to the wider range in x_i .

Battery electric vehicle application, scenario 3

- The final scenario we consider is identical to BEV scenario 2, except that we simulate a changing total capacity.
- The slope of the total capacity curve is chosen to be -0.01 Ah per measurement update, and $\gamma = 0.98$ was used.
- Representative results of this scenario are presented below.



- Once again, WLS fails and TLS is uncertain of its estimate for nearly 100 updates.
- However, TLS does recover and do quite well. The AWTLS method gives the best results.

4.18: Discussion of simulations

- The simulation results have illustrated a few key properties of the four methods we have discussed to estimate total capacity:
 - Noise on the SOC estimates must be considered in order to properly estimate battery total capacity.
 - ◆ Least squares, weighted least squares, and other similar methods simply fail.
 - ◆ They give biased estimates of total capacity, with unreliable error bounds.
 - ◆ Methods related to total least squares, where noises on the SOC estimates are explicitly recognized and incorporated in the calculations, are required for reliable total capacity estimation.
 - WTLS, in principle always gives the best results.
 - ◆ However, we have seen that in practice that the TLS and AWTLS methods can give better results because they can be initialized with a nominal capacity estimate.
 - ◆ Furthermore, since TLS and AWTLS give nice recursive solutions, one of them should always be used instead of WTLS.
 - If the measurement update interval m_i is constant, and therefore $\sigma_{x_i} = k\sigma_{y_i}$ for all measurements, TLS and AWTLS give identical results.
 - ◆ Therefore, the simpler TLS method is preferred.
 - ◆ However, if $\sigma_{x_i} \neq k\sigma_{y_i}$, the AWTLS method gives better results than TLS, and sometimes TLS fails.
 - ◆ This is particularly important for the BEV application where total capacity estimate updates are done when charging the battery,

to yield a greatly improved total capacity estimate because of the reduction in σ_{x_i} due to knowing one SOC value exactly.

- ◆ AWTLS always gives results at least as good as the other methods.
- The noises that contribute to σ_{y_i} are assumed to be due to current sensor errors.
 - ◆ In practice, these can include gain errors, bias errors, noise errors, and nonlinear errors.
 - ◆ We have considered only the noise errors here.
 - ◆ Gain errors and nonlinear errors will bias all of the methods; however, we believe that the biased value of the total capacity estimate will be consistent with the perceived capacity of the battery pack if the same current sensor is used to compute the battery pack total capacity estimate and to monitor pack operations.
 - ◆ Bias error can be subtracted in a BEV setting if we can assume that the Coulombic efficiency of the cells is $\eta \approx 1$ by matching the discharged ampere hours from usage with the charged ampere hours.
- The output error bounds on the total capacity estimate, even with the optimum WTLS estimator, are larger than some might expect.
 - ◆ This underscores the need for a method that predicts not only the estimate, but also dynamic error bounds on the estimate, as do the ones proposed in this article.
 - ◆ Without dynamic error bounds, the user of the total capacity estimate has no idea how good or bad that estimate is.

- ◆ If the estimate is used to compute battery pack available energy, for example, the energy estimate may be overly optimistic or overly pessimistic, neither of which is acceptable.

Where from here?

- We have now looked at the fundamental state and parameter estimation problems that must be solved by a BMS.
- These estimates can be fed into energy and power estimation algorithms.
 - We have already seen some simple examples of this;
 - We will look at more advanced methods in chapter 7.
- Next, however, we look at the somewhat simpler, but still very important issue of balancing or equalizing the cells in a battery pack.

Appendix: Nonlinear EKF for parameter estimation

Nonlinear state-space model:

$$\theta_{k+1} = \theta_k + r_k,$$

$$d_k = h(x_k, u_k, \theta_k, e_k).$$

where r_k and e_k are independent Gaussian noise processes with means zero and $\bar{e}$, respectively, and having covariance matrices $\Sigma_{\tilde{r}}$ and $\Sigma_{\tilde{e}}$, respectively.

Definition: $\hat{C}_k^\theta = \left. \frac{dh(x_k, u_k, \theta, e_k)}{d\theta} \right|_{\theta=\hat{\theta}_k^-}$ $\hat{D}_k^\theta = \left. \frac{dh(x_k, u_k, \theta, e_k)}{de_k} \right|_{e_k=\bar{e}_k}$

Caution: Be careful to compute $\hat{C}_k^\theta$ using recursive chain rule described in notes!

Initialization: For $k = 0$, set

$$\hat{\theta}_0^+ = \mathbb{E}[\theta_0]$$

$$\Sigma_{\tilde{\theta},0}^+ = \mathbb{E}[(\theta_0 - \hat{\theta}_0^+)(\theta_0 - \hat{\theta}_0^+)^T].$$

$$\frac{dx_0}{d\theta} = 0, \text{ unless side information is available.}$$

Computation: For $k = 1, 2, \dots$ compute:

State estimate time update: $\hat{\theta}_k^- = \hat{\theta}_{k-1}^+.$

Error covariance time update: $\Sigma_{\tilde{\theta},k}^- = \Sigma_{\tilde{\theta},k-1}^+ + \Sigma_{\tilde{r}}.$

Kalman gain matrix: $L_k^\theta = \Sigma_{\tilde{\theta},k}^- (\hat{C}_k^\theta)^T [\hat{C}_k^\theta \Sigma_{\tilde{\theta},k}^- (\hat{C}_k^\theta)^T + \hat{D}_k^\theta \Sigma_{\tilde{e}} (\hat{D}_k^\theta)^T]^{-1}.$

State estimate meas. update: $\hat{\theta}_k^+ = \hat{\theta}_k^- + L_k^\theta [d_k - h(x_k, u_k, \hat{\theta}_k^-, \bar{e}_k)].$

Error covariance meas. update: $\Sigma_{\tilde{\theta},k}^+ = (I - L_k^\theta \hat{C}_k^\theta) \Sigma_{\tilde{\theta},k}^-.$

Appendix: Nonlinear SPKF for parameter estimation

Nonlinear state-space model:

$$\theta_{k+1} = \theta_k + r_k,$$

$$d_k = h(x_k, u_k, \theta_k, e_k).$$

where r_k and e_k are independent Gaussian noise processes with means zero and $\bar{e}$, respectively, and having covariance matrices $\Sigma_{\tilde{r}}$ and $\Sigma_{\tilde{e}}$, respectively.

Definitions: Let

$$\theta_k^a = [\theta_k^T, e_k^T]^T, \quad \mathcal{W}_k^a = [(\mathcal{W}_k^\theta)^T, (\mathcal{W}_k^e)^T]^T, \quad p = 2 \times \dim(\theta_k^a).$$

Initialization: For $k = 0$, set

$$\begin{aligned} \hat{\theta}_0^+ &= \mathbb{E}[\theta_0] & \hat{\theta}_0^{a,+} &= \mathbb{E}[\theta_0^a] = [(\hat{\theta}_0^+)^T, \bar{e}]^T. \\ \Sigma_{\tilde{\theta},0}^+ &= \mathbb{E}[(\theta_0 - \hat{\theta}_0^+)(\theta_0 - \hat{\theta}_0^+)^T] & \Sigma_{\tilde{\theta},0}^{a,+} &= \mathbb{E}[(\theta_0^a - \hat{\theta}_0^{a,+})(\theta_0^a - \hat{\theta}_0^{a,+})^T] \\ & & &= \text{diag}(\Sigma_{\tilde{\theta},0}^+, \Sigma_{\tilde{e}}). \end{aligned}$$

Computation: For $k = 1, 2, \dots$ compute:

State estimate time update: $\hat{\theta}_k^- = \hat{\theta}_{k-1}^+.$

Error covariance time update: $\Sigma_{\tilde{\theta},k}^- = \Sigma_{\tilde{\theta},k-1}^+ + \Sigma_{\tilde{r}}.$

Output estimate: $\mathcal{W}_k^{a,-} = \left\{ \hat{\theta}_k^{a,-}, \hat{\theta}_k^{a,-} + \gamma \sqrt{\Sigma_{\tilde{\theta},k}^{a,-}}, \hat{\theta}_k^{a,-} - \gamma \sqrt{\Sigma_{\tilde{\theta},k}^{a,-}} \right\}.$

$$\mathcal{D}_{k,i} = h(x_k, u_k, \mathcal{W}_{k,i}^{\theta,-}, \mathcal{W}_{k,i}^{e,-}).$$

$$\hat{d}_k = \sum_{i=0}^p \alpha_i^{(m)} \mathcal{D}_{k,i}.$$

Computation (continued):

Estimator gain matrix:

$$\Sigma_{\tilde{d},k} = \sum_{i=0}^p \alpha_i^{(c)} (\mathcal{D}_{k,i} - \hat{d}_k)(\mathcal{D}_{k,i} - \hat{d}_k)^T.$$

$$\Sigma_{\tilde{\theta}\tilde{d},k}^- = \sum_{i=0}^p \alpha_i^{(c)} (\mathcal{W}_{k,i}^{\theta,-} - \hat{\theta}_k^-)(\mathcal{D}_{k,i} - \hat{d}_k)^T.$$

$$L_k^\theta = \Sigma_{\tilde{\theta}\tilde{d},k}^- \Sigma_{\tilde{d},k}^{-1}.$$

State estimate meas. update: $\hat{\theta}_k^+ = \hat{\theta}_k^- + L_k^\theta (d_k - \hat{d}_k).$

Error covariance meas. update: $\Sigma_{\tilde{\theta},k}^+ = \Sigma_{\tilde{\theta},k}^- - L_k^\theta \Sigma_{\tilde{d},k} (L_k^\theta)^T.$

Appendix: Joint EKF for state and parameter estimation

State-space model:

$$\begin{bmatrix} x_k \\ \theta_k \end{bmatrix} = \begin{bmatrix} f(x_{k-1}, u_{k-1}, w_{k-1}, \theta_{k-1}) \\ \theta_{k-1} + r_{k-1} \end{bmatrix} \quad \text{or} \quad \begin{aligned} \mathbb{X}_k &= \mathcal{F}(\mathbb{X}_{k-1}, u_{k-1}, \mathbb{W}_{k-1}) \\ z_k &= h(\mathbb{X}_k, u_k, v_k), \\ z_k &= h(x_k, u_k, v_k, \theta_k), \end{aligned}$$

where w_k , r_k , and v_k are independent, Gaussian noise processes of covariance matrices Σ_w , Σ_r , and Σ_v , respectively. For brevity, we let $\mathbb{X}_k = [x_k^T, \theta_k^T]^T$, $\mathbb{W}_k = [w_k^T, r_k^T]^T$ and $\Sigma_{\tilde{\mathbb{W}}} = \text{diag}(\Sigma_w, \Sigma_r)$.

Definitions:

$$\begin{aligned} \hat{A}_k &= \left. \frac{d\mathcal{F}(\mathbb{X}_k, u_k, \mathbb{W}_k)}{d\mathbb{X}_k} \right|_{\mathbb{X}_k = \hat{\mathbb{X}}_k^+} & \hat{B}_k &= \left. \frac{d\mathcal{F}(\mathbb{X}_k, u_k, \mathbb{W}_k)}{d\mathbb{W}_k} \right|_{\mathbb{W}_k = \bar{\mathbb{W}}_k} \\ \hat{C}_k &= \left. \frac{dh(\mathbb{X}_k, u_k, v_k)}{d\mathbb{X}_k} \right|_{\mathbb{X}_k = \hat{\mathbb{X}}_k^-} & \hat{D}_k &= \left. \frac{dh(\mathbb{X}_k, u_k, v_k)}{dv_k} \right|_{v_k = \bar{v}_k}. \end{aligned}$$

Initialization: For $k = 0$, set

$$\begin{aligned} \hat{\mathbb{X}}_0^+ &= \mathbb{E}[\mathbb{X}_0] \\ \Sigma_{\tilde{\mathbb{X}},0}^+ &= \mathbb{E}[(\mathbb{X}_0 - \hat{\mathbb{X}}_0^+)(\mathbb{X}_0 - \hat{\mathbb{X}}_0^+)^T] \end{aligned}$$

Computation: For $k = 1, 2, \dots$ compute:

$$\begin{aligned} \text{State estimate time update:} \quad & \hat{\mathbb{X}}_k^- = \mathcal{F}(\hat{\mathbb{X}}_{k-1}^+, u_{k-1}, \bar{\mathbb{W}}_{k-1}). \\ \text{Error covariance time update:} \quad & \Sigma_{\tilde{\mathbb{X}},k}^- = \hat{A}_{k-1} \Sigma_{\tilde{\mathbb{X}},k-1}^+ \hat{A}_{k-1}^T + \hat{B}_{k-1} \Sigma_{\tilde{\mathbb{W}}} \hat{B}_{k-1}^T. \\ \text{Output estimate:} \quad & \hat{z}_k = h(\hat{\mathbb{X}}_k^-, u_k, \bar{v}_k). \\ \text{Estimator gain matrix:} \quad & L_k = \Sigma_{\tilde{\mathbb{X}},k}^- \hat{C}_k^T [\hat{C}_k \Sigma_{\tilde{\mathbb{X}},k}^- \hat{C}_k^T + \hat{D}_k \Sigma_{\tilde{v}} \hat{D}_k^T]^{-1}. \\ \text{State estimate meas. update:} \quad & \hat{\mathbb{X}}_k^+ = \hat{\mathbb{X}}_k^- + L_k (z_k - \hat{z}_k). \\ \text{Error covariance meas. update:} \quad & \Sigma_{\tilde{\mathbb{X}},k}^+ = (I - L_k \hat{C}_k) \Sigma_{\tilde{\mathbb{X}},k}^-. \end{aligned}$$

Appendix: Dual EKF for state and parameter estimation

Nonlinear state-space models:

$$\begin{aligned} x_{k+1} &= f(x_k, u_k, \theta_k, w_k) & \theta_{k+1} &= \theta_k + r_k, \\ z_k &= h(x_k, u_k, \theta_k, v_k) & \text{and} & \\ d_k &= h(x_k, u_k, \theta_k, e_k). \end{aligned}$$

where w_k , v_k , r_k and e_k are independent Gaussian noise processes with means $\bar{w}$, $\bar{v}$, $\bar{r}$, and $\bar{e}$ and covariace matrices $\Sigma_{\bar{w}}$, $\Sigma_{\bar{v}}$, $\Sigma_{\bar{r}}$ and $\Sigma_{\bar{e}}$, respectively.

Definitions:

$$\begin{aligned} \hat{A}_k &= \left. \frac{df(x_k, u_k, \hat{\theta}_k^-, w_k)}{dx_k} \right|_{x_k = \hat{x}_k^+} & \hat{B}_k &= \left. \frac{df(x_k, u_k, \hat{\theta}_k^-, w_k)}{dw_k} \right|_{w_k = \bar{w}} \\ \hat{C}_k^x &= \left. \frac{dh(x_k, u_k, \hat{\theta}_k^-, v_k)}{dx_k} \right|_{x_k = \hat{x}_k^-} & \hat{D}_k^x &= \left. \frac{dh(x_k, u_k, \hat{\theta}_k^-, v_k)}{dv_k} \right|_{v_k = \bar{v}} \\ \hat{C}_k^\theta &= \left. \frac{dh(\hat{x}_k^-, u_k, \theta, e_k)}{d\theta} \right|_{\theta = \hat{\theta}_k^-} & \hat{D}_k^\theta &= \left. \frac{dh(\hat{x}_k^-, u_k, \theta, e_k)}{de_k} \right|_{e_k = \bar{e}} \end{aligned}$$

Initialization: For $k = 0$, set

$$\begin{aligned} \hat{\theta}_0^+ &= \mathbb{E}[\theta_0], & \Sigma_{\hat{\theta},0}^+ &= \mathbb{E}[(\theta_0 - \hat{\theta}_0^+)(\theta_0 - \hat{\theta}_0^+)^T]. \\ \hat{x}_0^+ &= \mathbb{E}[x_0], & \Sigma_{\hat{x},0}^+ &= \mathbb{E}[(x_0 - \hat{x}_0^+)(x_0 - \hat{x}_0^+)^T]. \end{aligned}$$

Computation: For $k = 1, 2, \dots$ compute:

Time update for the weight filter: $\hat{\theta}_k^- = \hat{\theta}_{k-1}^+$

$$\Sigma_{\hat{\theta},k}^- = \Sigma_{\hat{\theta},k-1}^+ + \Sigma_{\bar{r}}$$

Time update for the state filter: $\hat{x}_k^- = f(\hat{x}_{k-1}^+, u_{k-1}, \hat{\theta}_k^-, \bar{w})$

$$\Sigma_{\hat{x},k}^- = \hat{A}_{k-1} \Sigma_{\hat{x},k-1}^+ \hat{A}_{k-1}^T + \hat{B}_{k-1} \Sigma_{\bar{w}} \hat{B}_{k-1}^T$$

Meas. update for the state filter: $L_k^x = \Sigma_{\hat{x},k}^- (\hat{C}_k^x)^T [\hat{C}_k^x \Sigma_{\hat{x},k}^- (\hat{C}_k^x)^T + \hat{D}_k^x \Sigma_{\bar{v}} (\hat{D}_k^x)^T]^{-1}$

$$\hat{x}_k^+ = \hat{x}_k^- + L_k^x [z_k - h(\hat{x}_k^-, u_k, \hat{\theta}_k^-, \bar{v})]$$

$$\Sigma_{\hat{x},k}^+ = (I - L_k^x \hat{C}_k^x) \Sigma_{\hat{x},k}^-$$

Computation (cont.): For $k = 1, 2, \dots$ compute:

Meas. update for the weight filter:

$$L_k^\theta = \Sigma_{\tilde{\theta},k}^- (\hat{C}_k^\theta)^T [\hat{C}_k^\theta \Sigma_{\tilde{\theta},k}^- (\hat{C}_k^\theta)^T + \hat{D}_k^\theta \Sigma_{\tilde{e}} (\hat{D}_k^\theta)^T]^{-1}$$

$$\hat{\theta}_k^+ = \hat{\theta}_k^- + L_k^\theta [z_k - h(\hat{x}_k^-, u_k, \hat{\theta}_k^-, \bar{e})]$$

$$\Sigma_{\tilde{\theta},k}^+ = (I - L_k^\theta \hat{C}_k^\theta) \Sigma_{\tilde{\theta},k}^-$$

Appendix: Joint SPKF for state and parameter estimation

State-space model:

$$\begin{bmatrix} x_k \\ \theta_k \end{bmatrix} = \begin{bmatrix} f(x_{k-1}, u_{k-1}, w_{k-1}, \theta_{k-1}) \\ \theta_{k-1} + r_{k-1} \end{bmatrix} \quad \text{or} \quad \begin{aligned} \mathbb{X}_k &= \mathcal{F}(\mathbb{X}_{k-1}, u_{k-1}, \mathbb{W}_{k-1}) \\ z_k &= h(\mathbb{X}_k, u_k, v_k), \\ z_k &= h(x_k, u_k, v_k, \theta_k), \end{aligned}$$

where w_k , r_k , and v_k are independent, Gaussian noise processes with means $\bar{w}$, $\bar{r}$, and $\bar{v}$, and covariance matrices $\Sigma_{\tilde{w}}$, $\Sigma_{\tilde{r}}$, and $\Sigma_{\tilde{v}}$, respectively. For brevity, we let $\mathbb{X}_k = [x_k^T, \theta_k^T]^T$, $\mathbb{W}_k = [w_k^T, r_k^T]^T$ and $\Sigma_{\tilde{\mathbb{W}}} = \text{diag}(\Sigma_{\tilde{w}}, \Sigma_{\tilde{r}})$.

Definitions: Let

$$\mathbb{X}_k^a = [\mathbb{X}_k^T, \mathbb{W}_k^T, v_k^T]^T, \quad \mathcal{X}_k^a = [(\mathcal{X}_k^{\mathbb{X}})^T, (\mathcal{X}_k^{\mathbb{W}})^T, (\mathcal{X}_k^v)^T]^T, \quad p = 2 \times \dim(\mathbb{X}_k^a).$$

Initialization: For $k = 0$, set

$$\begin{aligned} \hat{\mathbb{X}}_0^+ &= \mathbb{E}[\mathbb{X}_0] & \hat{\mathbb{X}}_0^{a,+} &= \mathbb{E}[\mathbb{X}_0^a] = [(\hat{\mathbb{X}}_0^+)^T, \bar{\mathbb{W}}, \bar{v}]^T. \\ \Sigma_{\tilde{\mathbb{X}},0}^+ &= \mathbb{E}[(\mathbb{X}_0 - \hat{\mathbb{X}}_0^+)(\mathbb{X}_0 - \hat{\mathbb{X}}_0^+)^T] & \Sigma_{\tilde{\mathbb{X}},0}^{a,+} &= \mathbb{E}[(\mathbb{X}_0^a - \hat{\mathbb{X}}_0^{a,+})(\mathbb{X}_0^a - \hat{\mathbb{X}}_0^{a,+})^T] \\ & & &= \text{diag}(\Sigma_{\tilde{\mathbb{X}},0}^+, \Sigma_{\tilde{\mathbb{W}}}, \Sigma_{\tilde{v}}). \end{aligned}$$

Computation: For $k = 1, 2, \dots$ compute:

State estimate time update: $\mathcal{X}_{k-1}^{a,+} = \left\{ \hat{\mathbb{X}}_{k-1}^{a,+}, \hat{\mathbb{X}}_{k-1}^{a,+} + \gamma \sqrt{\Sigma_{\hat{\mathbb{X}},k-1}^{a,+}}, \hat{\mathbb{X}}_{k-1}^{a,+} - \gamma \sqrt{\Sigma_{\hat{\mathbb{X}},k-1}^{a,+}} \right\}.$

$$\mathcal{X}_{k,i}^{\mathbb{X},-} = \mathcal{F}(\mathcal{X}_{k-1,i}^{\mathbb{X},+}, u_{k-1}, \mathcal{X}_{k-1,i}^{\mathbb{W},+}).$$

$$\hat{\mathbb{X}}_k^- = \sum_{i=0}^p \alpha_i^{(m)} \mathcal{X}_{k,i}^{\mathbb{X},-}.$$

Error covariance time update: $\Sigma_{\hat{\mathbb{X}},k}^- = \sum_{i=0}^p \alpha_i^{(c)} (\mathcal{X}_{k,i}^{\mathbb{X},-} - \hat{\mathbb{X}}_k^-) (\mathcal{X}_{k,i}^{\mathbb{X},-} - \hat{\mathbb{X}}_k^-)^T.$

Output estimate: $\mathcal{Z}_{k,i} = h(\mathcal{X}_{k,i}^{\mathbb{X},-}, u_k, \mathcal{X}_{k-1,i}^{v,+}).$

$$\hat{\mathbf{z}}_k = \sum_{i=0}^p \alpha_i^{(m)} \mathcal{Z}_{k,i}.$$

Estimator gain matrix: $\Sigma_{\tilde{\mathbf{z}},k} = \sum_{i=0}^p \alpha_i^{(c)} (\mathcal{Z}_{k,i} - \hat{\mathbf{z}}_k) (\mathcal{Z}_{k,i} - \hat{\mathbf{z}}_k)^T.$

$$\Sigma_{\hat{\mathbb{X}}\tilde{\mathbf{z}},k}^- = \sum_{i=0}^p \alpha_i^{(c)} (\mathcal{X}_{k,i}^{\mathbb{X},-} - \hat{\mathbb{X}}_k^-) (\mathcal{Z}_{k,i} - \hat{\mathbf{z}}_k)^T.$$

$$L_k = \Sigma_{\hat{\mathbb{X}}\tilde{\mathbf{z}},k}^- \Sigma_{\tilde{\mathbf{z}},k}^{-1}.$$

State estimate meas. update: $\hat{\mathbb{X}}_k^+ = \hat{\mathbb{X}}_k^- + L_k (\mathbf{z}_k - \hat{\mathbf{z}}_k).$

Error covariance meas. update: $\Sigma_{\hat{\mathbb{X}},k}^+ = \Sigma_{\hat{\mathbb{X}},k}^- - L_k \Sigma_{\tilde{\mathbf{z}},k} L_k^T.$

Appendix: Dual SPKF for state and parameter estimation

Nonlinear state-space models:

$$\begin{aligned} x_k &= f(x_{k-1}, u_{k-1}, w_{k-1}, \theta_{k-1}) & \theta_k &= \theta_{k-1} + r_{k-1}, \\ z_k &= h(x_k, u_k, v_k, \theta_k) & \text{and} & & d_k &= h(f(x_{k-1}, u_{k-1}, \bar{w}_{k-1}, \theta_{k-1}), u_k, \bar{v}_k, \theta_{k-1}, e_k). \end{aligned}$$

where w_k , v_k , r_k and e_k are independent, Gaussian noise processes with means $\bar{w}$, $\bar{v}$, $\bar{r}$, and $\bar{e}$, and covariance matrices $\Sigma_{\bar{w}}$, $\Sigma_{\bar{v}}$, $\Sigma_{\bar{r}}$ and $\Sigma_{\bar{e}}$, respectively.

Definitions: $x_k^a = [x_k^T, w_k^T, v_k^T]^T$, $\mathcal{X}_k^a = [(\mathcal{X}_k^x)^T, (\mathcal{X}_k^w)^T, (\mathcal{X}_k^v)^T]^T$, $p = 2 \times \dim(x_k^a)$.

Initialization: For $k = 0$, set

$$\begin{aligned} \hat{\theta}_0^+ &= \mathbb{E}[\theta_0], & \Sigma_{\hat{\theta},0}^+ &= \mathbb{E}[(\theta_0 - \hat{\theta}_0^+)(\theta_0 - \hat{\theta}_0^+)^T], \\ \hat{x}_0^+ &= \mathbb{E}[x_0], & \hat{x}_0^{a,+} &= \mathbb{E}[x_0^a] = [(\hat{x}_0^+)^T, \bar{w}, \bar{v}]^T, \\ \Sigma_{\hat{x},0}^+ &= \mathbb{E}[(x_0 - \hat{x}_0^+)(x_0 - \hat{x}_0^+)^T], & \Sigma_{\hat{x},0}^{a,+} &= \mathbb{E}[(x_0^a - \hat{x}_0^{a,+})(x_0^a - \hat{x}_0^{a,+})^T] \\ & & &= \text{diag}(\Sigma_{\hat{x},0}^+, \Sigma_w, \Sigma_v). \end{aligned}$$

Computation: For $k = 1, 2, \dots$ compute:

$$\begin{aligned} \text{Parameter estimate time update:} & \quad \hat{\theta}_k^- = \hat{\theta}_{k-1}^+, \\ \text{Parameter covariance time update:} & \quad \Sigma_{\hat{\theta},k}^- = \Sigma_{\hat{\theta},k-1}^+ + \Sigma_{\bar{r}}, \\ \text{State estimate time update:} & \quad \mathcal{X}_{k-1}^{a,+} = \left\{ \hat{x}_{k-1}^{a,+}, \hat{x}_{k-1}^{a,+} + \gamma \sqrt{\Sigma_{\hat{x},k-1}^{a,+}}, \hat{x}_{k-1}^{a,+} - \gamma \sqrt{\Sigma_{\hat{x},k-1}^{a,+}} \right\}, \\ & \quad \mathcal{X}_{k,i}^{x,-} = f(\mathcal{X}_{k-1,i}^{x,+}, u_{k-1}, \mathcal{X}_{k-1,i}^{w,+}, \hat{\theta}_k^-), \\ & \quad \hat{x}_k^- = \sum_{i=0}^p \alpha_i^{(m)} \mathcal{X}_{k,i}^{x,-}, \\ \text{State covariance time update:} & \quad \Sigma_{\hat{x},k}^- = \sum_{i=0}^p \alpha_i^{(c)} (\mathcal{X}_{k,i}^{x,-} - \hat{x}_k^-)(\mathcal{X}_{k,i}^{x,-} - \hat{x}_k^-)^T. \end{aligned}$$

(continued...)

Computation (continued): For $k = 1, 2, \dots$ compute:

<i>Output estimate, parameter filter:</i>	$\mathcal{W}_k = \left\{ \hat{\theta}_k^-, \hat{\theta}_k^- + \gamma \sqrt{\Sigma_{\tilde{\theta},k}^-}, \hat{\theta}_k^- - \gamma \sqrt{\Sigma_{\tilde{\theta},k}^-} \right\}.$ $\mathcal{D}_{k,i} = h(f(\hat{x}_{k-1}^+, u_{k-1}, \bar{w}_{k-1}, \mathcal{W}_{k,i}), u_k, \bar{v}_k, \mathcal{W}_{k,i}).$ $\hat{d}_k = \sum_{i=0}^p \alpha_i^{(m)} \mathcal{D}_{k,i}$
<i>Output estimate, state filter:</i>	$\mathcal{Z}_{k,i} = h(\mathcal{X}_{k,i}^{x,-}, u_k, \mathcal{X}_{k-1,i}^{v,+}, \hat{\theta}_k^-).$ $\hat{z}_k = \sum_{i=0}^p \alpha_i^{(m)} \mathcal{Z}_{k,i}.$
<i>State filter gain matrix:</i>	$\Sigma_{\tilde{z},k} = \sum_{i=0}^p \alpha_i^{(c)} (\mathcal{Z}_{k,i} - \hat{z}_k) (\mathcal{Z}_{k,i} - \hat{z}_k)^T.$ $\Sigma_{\tilde{x}\tilde{z},k} = \sum_{i=0}^p \alpha_i^{(c)} (\mathcal{X}_{k,i}^{x,-} - \hat{x}_k^-) (\mathcal{Z}_{k,i} - \hat{z}_k)^T.$ $L_k^x = \Sigma_{\tilde{x}\tilde{z},k}^- \Sigma_{\tilde{z},k}^{-1}.$
<i>Parameter filter gain matrix:</i>	$\Sigma_{\tilde{d},k} = \sum_{i=0}^p \alpha_i^{(c)} (\mathcal{D}_{k,i} - \hat{d}_k) (\mathcal{D}_{k,i} - \hat{d}_k)^T.$ $\Sigma_{\tilde{\theta}\tilde{d},k} = \sum_{i=0}^p \alpha_i^{(c)} (\mathcal{W}_{k,i} - \hat{\theta}_k^-) (\mathcal{D}_{k,i} - \hat{d}_k)^T.$ $L_k^\theta = \Sigma_{\tilde{\theta}\tilde{d},k}^- \Sigma_{\tilde{d},k}^{-1}.$
<i>State estimate measurement update:</i>	$\hat{x}_k^+ = \hat{x}_k^- + L_k^x (z_k - \hat{z}_k).$
<i>State covariance measurement update:</i>	$\Sigma_{\tilde{x},k}^+ = \Sigma_{\tilde{x},k}^- - L_k^x \Sigma_{\tilde{z},k} (L_k^x)^T.$
<i>Parameter estimate meas. update:</i>	$\hat{\theta}_k^+ = \hat{\theta}_k^- + L_k^\theta (z_k - \hat{d}_k).$
<i>Parameter covariance meas. update:</i>	$\Sigma_{\tilde{\theta},k}^+ = \Sigma_{\tilde{\theta},k}^- - L_k^\theta \Sigma_{\tilde{d},k} (L_k^\theta)^T.$

Appendix: Code to simulate TLS methods

- This first code segment, `runSixScenarios.m`, sets up and runs the six different scenarios

```
% Run the six scenarios in the notes for the different capacity
% estimation methods.

scenarios = ones([6 1]); % set to 1 to run a test scenario, 0 to skip

% Scenario 1:
if scenarios(1),
    Q0 = 10;
    maxI = 30*Q0;          % must be able to measure current up to +/- maxI
    precisionI = 1024; % 10-bit precision on current sensor
    slope = 0;
    Qnom = 0;
    xmax = 0.2;
    xmin = -xmax;
    m = 300;
    theCase = 1;
    socnoise = sqrt(2)*0.01;
    gamma = 1;
    plotParams.title = 'HEV Scenario 1';
    plotParams.fileName = 'Figure 3';
    plotParams.ytick = 9.5:0.1:10.5;
    RandStream.setGlobalStream(RandStream('mt19937ar','seed',8));

    runScenario
end

% Scenario 2:
if scenarios(2),
    Q0 = 10;
    maxI = 30*Q0;          % must be able to measure current up to +/- maxI
    precisionI = 1024; % 10-bit precision on current sensor
    slope = 0;
    Qnom = 0.99*Q0;
    xmax = 0.2;
    xmin = -xmax;
    m = 300;
    theCase = 1;
```

```

socnoise = sqrt(2)*0.01;
gamma = 1;
plotParams.title = 'HEV Scenario 2';
plotParams.fileName = 'Figure 4';
plotParams.ytick = 9.5:0.1:10.5;
RandStream.setGlobalStream(RandStream('mt19937ar','seed',8));

runScenario
end

% Scenario 3:
if scenarios(3),
    Q0 = 10;
    maxI = 30*Q0;           % must be able to measure current up to +/- maxI
    precisionI = 1024; % 10-bit precision on current sensor
    slope = -0.001;
    Qnom = 0.99*Q0;
    xmax = 0.2;
    xmin = -xmax;
    m = 300;
    theCase = 1;
    socnoise = sqrt(2)*0.01;
    gamma = 0.99;
    plotParams.title = 'HEV Scenario 3';
    plotParams.fileName = 'Figure 5';
    plotParams.ytick = 8.7:0.2:10.5;
    RandStream.setGlobalStream(RandStream('mt19937ar','seed',8));

    runScenario
end

% Scenario 4:
if scenarios(4),
    Q0 = 100;
    maxI = 5*Q0;           % must be able to measure current up to +/- maxI
    precisionI = 1024; % 10-bit precision on current sensor
    slope = 0;
    Qnom = 0.99*Q0;
    xmax = 0.4;
    xmin = -xmax;
    theCase = 1; m = 7200;
    socnoise = sqrt(2)*0.01;

```

```

gamma = 1;
plotParams.title = 'EV Scenario 1';
plotParams.fileName = 'Figure 6';
plotParams.ytick = 97:1:103;
RandStream.setGlobalStream(RandStream('mt19937ar','seed',5));

runScenario
end

% Scenario 5:
if scenarios(5),
    Q0 = 100;
    maxI = 5*Q0;           % must be able to measure current up to +/- maxI
    precisionI = 1024; % 10-bit precision on current sensor
    slope = 0;
    Qnom = 0.99*Q0;
    xmax = 0.8;
    xmin = -xmax;
    theCase = 2; mode = 0.5; sigma = 0.6;
    socnoise = 0.01;
    gamma = 1;
    plotParams.title = 'EV Scenario 2';
    plotParams.fileName = 'Figure 8';
    plotParams.ytick = 98:0.5:101;
    RandStream.setGlobalStream(RandStream('mt19937ar','seed',4));

    runScenario
end

% Scenario 6:
if scenarios(6),
    Q0 = 100;
    maxI = 5*Q0;           % must be able to measure current up to +/- maxI
    precisionI = 1024; % 10-bit precision on current sensor
    slope = -0.01;
    Qnom = 0.99*Q0;
    xmax = 0.8;
    xmin = -xmax;
    theCase = 2; mode = 0.5; sigma = 0.6;
    socnoise = 0.01;
    gamma = 0.98;
    plotParams.title = 'EV Scenario 3';

```

```

plotParams.fileName = 'Figure 9';
plotParams.ytick = 89:1:103;
RandStream.setGlobalStream(RandStream('mt19937ar','seed',5));

runScenario
end

```

- This second code segment, `runscenario.m`, runs an individual scenario, and plots results

```

% Run a particular scenario and plot results
% (set up by "runSixScenarios.m")
if ~exist('FIGURES','dir'),
    mkdir FIGURES
end

% Make up some data.
n = 1000;
Q = (Q0+slope*(1:n))';
x = ((xmax-xmin)*rand(n,1)+xmin);
y = Q.*x;

% Add in some noise to both variables.
binsize = 2*maxI/precisionI;
rn1 = ones(n,1);
if theCase == 1,
    rn2 = rn1;
    sy = binsize*sqrt(m/12)/3600*rn2;
else
    mu = log(mode)+sigma^2;
    m = 3600*lognrnd(mu,sigma,n,1);
    if 1,
        figure(3); clf; hold on;
        plot(0:0.01:3,lognpdf(0:0.01:3,mu,sigma),'k');
        xlabel('Time (hours)');
        ylabel('PDF value');
        title('PDF of drive cycle durations for EV scenarios');
        box on;
        print -deps2 FIGURES/Figure7.eps
    end
    sy = binsize*sqrt(m/12)/3600;
end

```

```

sx = socnoise*rn1;

x = x + sx.*randn(n,1);
y = y + sy.*randn(n,1);

% - column 1 = WLS - weighted, recursive
% - column 2 = WTLS - weighted, but not recursive
% - column 3 = TLS - recursive weighted, but using SigmaX(1) and
%           SigmaY(1) only (i.e., simplified method using c0 and c1)
% - column 4 = AWTLS - recursive and weighted
[Qhat,SigmaQ,Fit] = xLSalgos(x,y,sx.^2,sy.^2,gamma,Qnom);
Qrep = repmat(Q,1,size(Qhat,2));
fileName = plotParams.fileName; fileName(fileName == ' ') = [];

figure(1); clf; hold on;
plot(Qhat);
xlabel('Algorithm update index');
ylabel('Capacity estimate (Ah)');
title(sprintf('%s: Capacity Estimates with Error Bounds',...
              plotParams.title));
legend('WLS','WTLS','TLS','AWTLS','location','northeast');
box on;
plot(Qhat+3*sqrt(SigmaQ),'linewidth',0.5);
plot(Qhat-3*sqrt(SigmaQ),'linewidth',0.5);
plot(Qhat);
plot(1:n,Q,'k:','linewidth',1);
ylim([min(plotParams.ytick) max(plotParams.ytick)]);
set(gca,'ytick',plotParams.ytick);
print(sprintf('FIGURES/%s_a.eps',fileName),'-deps2c');

figure(2); clf; hold on
plot(Fit)
xlabel('Algorithm update index');
ylabel('Goodness of fit');
title(sprintf('%s: Goodness of fit for each method',plotParams.title))
legend('WLS','WTLS','TLS','AWTLS','location','east');
box on; ylim([-0.02 1.02]);
print(sprintf('FIGURES/%s_b.eps',fileName),'-deps2c');

drawnow

```

- This third code segment, `xLSalgos.m`, does the real work

```

% Tests the recursive performance of the xLS algorithms on a particular
% dataset
% [Qhat, SigmaQ, Fit] = xLSalgos(measX, measY, SigmaX, SigmaY, gamma, Qnom)
%   - measX = noisy z(2)-z(1)
%   - measY = noisy integral(i(t)/3600 dt)
%   - SigmaX = variance of X
%   - SigmaY = variance of Y
%   - gamma = geometric forgetting factor (gamma = 1 for perfect memory)
%   - Qnom = nominal value of Q: if nonzero, used to initialize recursions
%
%   - Qhat = estimate of capacity at every time step
%   - column 1 = WLS - weighted, recursive
%   - column 2 = WTLS - weighted, but not recursive
%   - column 3 = SCTLS - scaled confidence TLS; recursive and weighted,
%               but using SigmaX(1) and SigmaY(1) only to determine
%               factor by which all SigmaX and SigmaY are assumed to be
%               related
%   - column 4 = AWTLS - recursive and weighted
%   - SigmaQ = variance of Q, computed via Hessian method (columns
%               correspond to methods in the same way as for Qhat)
%   - Fit = goodness of fit metric for each method (columns
%           correspond to methods in the same way as for Qhat)
function [Qhat, SigmaQ, Fit] = xLSalgos(measX, measY, SigmaX, SigmaY, gamma, Qnom)
% Reserve some memory
Qhat = zeros(length(measX), 4); SigmaQ = Qhat; Fit = Qhat;
K = sqrt(SigmaX(1)/SigmaY(1));

% Initialize some variables used for the recursive methods
c1 = 0; c2 = 0; c3 = 0; c4 = 0; c5 = 0; c6 = 0;
C1 = 0; C2 = 0; C3 = 0; C4 = 0; C5 = 0; C6 = 0;
if Qnom ~= 0,
    c1 = 1/SigmaY(1); c2 = Qnom/SigmaY(1); c3 = Qnom^2/SigmaY(1);
    c4 = 1/SigmaX(1); c5 = Qnom/SigmaX(1); c6 = Qnom^2/SigmaX(1);
    C1 = 1/(K^2*SigmaY(1)); C2 = K*Qnom/(K^2*SigmaY(1));
    C3 = K^2*Qnom^2/(K^2*SigmaY(1));
    C4 = 1/SigmaX(1); C5 = K*Qnom/SigmaX(1); C6 = K^2*Qnom^2/SigmaX(1);
end

for iter = 1:length(measX),
    % Compute some variables used for the recursive methods
    c1 = gamma*c1 + measX(iter)^2/SigmaY(iter);
    c2 = gamma*c2 + measX(iter)*measY(iter)/SigmaY(iter);

```

```

c3 = gamma*c3 + measY(iter)^2/SigmaY(iter);
c4 = gamma*c4 + measX(iter)^2/SigmaX(iter);
c5 = gamma*c5 + measX(iter)*measY(iter)/SigmaX(iter);
c6 = gamma*c6 + measY(iter)^2/SigmaX(iter);

C1 = gamma*C1 + measX(iter)^2/(K^2*SigmaY(iter));
C2 = gamma*C2 + K*measX(iter)*measY(iter)/(K^2*SigmaY(iter));
C3 = gamma*C3 + K^2*measY(iter)^2/(K^2*SigmaY(iter));
C4 = gamma*C4 + measX(iter)^2/SigmaX(iter);
C5 = gamma*C5 + K*measX(iter)*measY(iter)/SigmaX(iter);
C6 = gamma*C6 + K^2*measY(iter)^2/SigmaX(iter);

% Method 1: WLS
Q = c2./c1; Qhat(iter,1) = Q;
H = 2*c1; SigmaQ(iter,1) = 2/H;
J = Q.^2.*c1 -2*Q.*c2 + c3;
Fit(iter,1) = gammainc(J/2, (iter-1)/2, 'upper');

% Method 2: WTLS -- not recursive -- implementation 1
% this implementation works, but is slower than FMINSEARCH method
% Q = zeros(size(measX));
% H = zeros(size(measX));
% for k = 1:length(measX),
%     x = measX(1:k);
%     y = measY(1:k);
%     sx = sqrt(SigmaX(1:k));
%     sy = sqrt(SigmaY(1:k));
%     Ctls2 = Qhat(k,1);
%     for kk = 1:10,
%         jacobian = sum((2*(Ctls2*x-y).*(Ctls2*y.*sx.^2+x.*sy.^2))./...
%             ((Ctls2^2*sx.^2+sy.^2).^2));
%         hessian = sum((2*sy.^4.*x.^2+sx.^4.*...
%             (6*Ctls2^2*y.^2-4*Ctls2^3*x.*y) - ...
%             sx.^2.*sy.^2.*...
%             (6*Ctls2^2*x.^2-12*Ctls2*x.*y+2*y.^2))./...
%             ((Ctls2^2*sx.^2+sy.^2).^3));
%         Ctls2 = Ctls2 - jacobian/hessian;
%     end
%     Q(k) = Ctls2;
%     H(k) = hessian;
% end
% Qhat(:,2) = Q;

```

```

% SigmaQ(:,2) = 2./H;

% Method 2: WTLS -- not recursive -- implementation 2
% This implementation uses FMINSEARCH, and is faster than method above
x = measX(1:iter); y = measY(1:iter);
sx = sqrt(SigmaX(1:iter)); sy = sqrt(SigmaY(1:iter));
g = flipud((gamma.^(0:(iter-1)))');
Q = fminsearch(@(q) sum(g.*(y-q*x).^2./(sx.^2*q^2+sy.^2)),...
               Qhat(iter,1));
H = 2*sum(g.*((sy.^4.*x.^2+sx.^4.*(3*Q^2*y.^2-2*Q^3*x.*y) - ...
              sx.^2.*sy.^2.*(3*Q^2*x.^2-6*Q*x.*y+y.^2))./...
              ((Q^2*sx.^2+sy.^2).^3)));
Qhat(iter,2) = Q;
SigmaQ(iter,2) = 2/H;
J = sum(g.*(y-Q*x).^2./(sx.^2*Q^2+sy.^2));
Fit(iter,2) = gammainc(J/2, (2*iter-1)/2, 'upper');

% Method 3: RTLS
Q = (-c1+K^2*c3+sqrt((c1-K^2*c3)^2+4*K^2*c2^2))/(2*K^2*c2);
Qhat(iter,3) = Q;
H = ((-4*K^4*c2)*Q^3+6*K^4*c3*Q^2+...
      (-6*c1+12*c2)*K^2*Q+2*(c1-K^2*c3))/(Q^2*K^2+1)^3;
SigmaQ(iter,3) = 2/H;
J = (Q^2*c1 -2*Q*c2 + c3)/(Q^2*K^2+1);
Fit(iter,3) = gammainc(J/2, (2*iter-1)/2, 'upper');

% % Method 4a: AWTLS without pre-scaling
% r = roots([c5 (-c1+2*c4-c6) (3*c2-3*c5) (c1-2*c3+c6) -c2]);
% r = r(r==conj(r));
% r = r(r>0);
% Jr = ((1./(r.^2+1).^2).*(r.^4*c4-2*c5*r.^3+(c1+c6)*r.^2-2*c2*r+c3))';
% Q = r(Jr==min(Jr));
% J = min(Jr);
% H = (2/(Q^2+1)^4)*...
%      (-2*c5*Q^5+(3*c1-6*c4+3*c6)*Q^4+(-12*c2+16*c5)*Q^3 ...
%      +(-8*c1+10*c3+6*c4-8*c6)*Q^2+(12*c2-6*c5)*Q+(c1-2*c3+c6));
% Qhat(iter,4) = Q;
% SigmaQ(iter,4) = 2/H;
% Fit(iter,4) = gammainc(J/2, (2*iter-1)/2, 'upper');

% Method 4b: AWTLS with pre-scaling
r = roots([C5 (-C1+2*C4-C6) (3*C2-3*C5) (C1-2*C3+C6) -C2]);

```

```

r = r(r==conj(r)); % discard complex-conjugate roots
r = r(r>0); % discard negative roots
Jr = ((1./(r.^2+1).^2).* (r.^4*C4-2*C5*r.^3+(C1+C6)*r.^2-2*C2*r+C3))';
J = min(Jr);
Q = r(Jr==J); % keep Q that minimizes cost function
H = (2/(Q^2+1)^4)*...
      (-2*C5*Q^5+(3*C1-6*C4+3*C6)*Q^4+(-12*C2+16*C5)*Q^3 ...
      +(-8*C1+10*C3+6*C4-8*C6)*Q^2+(12*C2-6*C5)*Q+(C1-2*C3+C6));
Qhat(iter,4) = Q/K;
SigmaQ(iter,4) = 2/H/K^2;
Fit(iter,4) = gammainc(J/2, (2*iter-1)'/2, 'upper');
end
Fit = real(Fit);
return

```

- Just for fun, testFerrari.m, an implementation of Ferrari's quartic root-finding code, to replace MATLAB's roots.m, better suited for a real-time embedded-systems implementation

```

% Tests Ferrari's method of solving a quartic equation for its roots
% e.g., testFerrari([1 -4 -1 16 -12]);
%       testFerrari([1 -14 44 -40 0]);
%       testFerrari([1 -12 54 -108 81]);
function testFerrari(R)
    A = R(1); B = R(2); C = R(3); D = R(4); E = R(5);

    clc; fprintf('Roots found using Ferrari's method:');
    quartic_Ferrari=sort(get_quartic_F(A,B,C,D,E))

    fprintf('Roots found using MATLAB's "roots" command:');
    % conj b/c sorted differently for whatever reason...
    roots_solution=conj(sort(roots([A,B,C,D,E])))

    fprintf('Norm of difference between solutions (should be small): ');
    fprintf('%g\n\n',norm(quartic_Ferrari - roots_solution));
end

%-----
function x = get_quartic_F(A,B,C,D,E)
% Solve a quartic equation using Ferrari's method
% see http://en.wikipedia.org/wiki/Quartic_equation

```

```

m_min = 1e-20;

alpha = C/A - 3*B^2/(8*A^2);
beta  = B^3/(8*A^3) - B*C/(2*A^2) + D/A;
gamma = -3*B^4/(256*A^4) + C*B^2/(16*A^3) - B*D/(4*A^2) + E/A;

% P,Q always real if A,B,C,D,E are real.
P = -alpha^2/12 - gamma;
Q = -alpha^3/108 + alpha*gamma/3 - beta^2/8;

% Now want to find roots([1 0 P Q]) and keep either (a) the only
% real root, r1, or (b) the maximum of the real roots, r1.
% Then y = -5/6*alpha + r1.
q = P/3; r = -Q/2;
delta = q^3+r^2;
if delta>0 % one real root, two imaginary
    s = nthroot(r+sqrt(delta),3);
    t = nthroot(r-sqrt(delta),3);
else
    rho = abs(r+1i*sqrt(-delta));
    if abs(rho)>0, theta = acos(r/rho); else theta = 0; end
    s = (rho)^(1/3)*exp(1i*theta/3);
    t = (rho)^(1/3)*exp(-1i*theta/3);
end
r = [(s+t) -0.5*(s+t)+sqrt(3)*(s-t)*1i/2,...
      -0.5*(s+t)-sqrt(3)*(s-t)*1i/2];
r = sort(r);
r = r(r==conj(r)); % keep real ones
r = max(r); % max seems to be necessary!
y = -5/6*alpha + r;

% W always real if A,B,C,D,E are real
W = real(sqrt(alpha + 2*y));

x(1) = -B/(4*A) + ( +W + sqrt(-(3*alpha + 2*y + 2*beta/(W + m_min))))/2;
x(2) = -B/(4*A) + ( -W + sqrt(-(3*alpha + 2*y - 2*beta/(W + m_min))))/2;
x(3) = -B/(4*A) + ( +W - sqrt(-(3*alpha + 2*y + 2*beta/(W + m_min))))/2;
x(4) = -B/(4*A) + ( -W - sqrt(-(3*alpha + 2*y - 2*beta/(W + m_min))))/2;
end

```